

POLITECNICO
MILANO 1863

Best Bike Paths – RASD Document

RASD document for the
Software Engineering 2 course assignment

Authors: **Alessandro Burato, Gabriele Crimella, Davide Faccini**

Teacher: Elisabetta Di Nitto
Academic Year: 2025-26

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.1.1 Goals . . . 1
1.2 Scope . . . 2
1.2.1 World Phenomena . . . 2
1.2.2 Machine Phenomena . . . 3
1.2.3 Shared Phenomena . . . 3
1.3 Definitions, Acronyms, Abbreviations . . . 4
1.4 Reference Documents . . . 4
1.4.1 Document Structure . . . 4

2 Overall Description **7**
2.1 Product perspective . . . 7
2.1.1 Scenarios . . . 7
2.1.2 Class diagram . . . 8
2.1.3 State diagrams . . . 9
2.2 Product functions . . . 10
2.3 User charateristic . . . 11
2.4 Assumptions, dependencies and constraints . . . 12
2.4.1 Domain assumptions . . . 12

3 Specific Requirements **15**
3.1 Functional Requirements . . . 15
3.1.1 Requirements . . . 15
3.1.2 Use Cases Diagrams . . . 18
3.1.3 Use Cases . . . 19
3.1.4 Mapping on Goals . . . 40
3.2 Performance Requirements . . . 45

3.3 Design Constraints . . . 47
3.3.1 Standard compliance . . . 47
3.3.2 Hardware limitations . . . 47
3.4 Software System Attributes . . . 48
3.4.1 Reliability . . . 48
3.4.2 Availability . . . 48
3.4.3 Security . . . 48
3.4.4 Maintainability . . . 49
3.4.5 Portability . . . 49

4 Formal Analysis Using Alloy **51**
4.1 Signatures . . . 51
4.1.1 Status . . . 51
4.1.2 Segment . . . 51
4.1.3 Path . . . 52
4.1.4 Statistics . . . 52
4.1.5 Trip . . . 52
4.1.6 RegisteredUser . . . 53
4.2 Facts . . . 53
4.2.1 Paths linked to Trips . . . 53
4.2.2 Statistics linked to Trips . . . 53
4.2.3 Trips linked to Users . . . 54
4.2.4 User has at least one Trip . . . 54
4.2.5 Published Trips are subset to Regisetered ones . . . 55
4.2.6 Start segments don't have backlinks . . . 55
4.2.7 Trips are personal . . . 55
4.2.8 Path coherence . . . 56
4.2.9 Path equality . . . 56
4.3 Static Model Graph . . . 57
4.3.1 Graphs and proof of consistency . . . 57
4.4 Merging of two paths . . . 60
4.4.1 Vars in signatures . . . 60
4.4.2 Merge predicate . . . 60
4.4.3 Example predicate . . . 61

5 Effort Spent **65**

6 References **67**

List of Figures **69**

1 Introduction

1.1. Purpose

The primary purpose of this project, initiated by a leading Bike Association, is to develop and deploy an advanced support system for the cycling community designed to thrive within a transport landscape dominated by cars.

The core idea is to foster a collaborative digital environment where cyclists can actively record, consult, and publish both their own and other users' riding paths.

This platform will serve as the definitive centralized resource for finding highly recommended routes tailored precisely to the user's specific experience, preferences, and safety considerations.

Ultimately, the system aims to significantly enhance the quality of the cycling experience by providing verified, community-rated routes, thereby connecting riders and making high-quality path discovery universally accessible

1.1.1. Goals

Here are the goals that BBP has to fulfill. BBP has to:

Goal	Description
G1	Recognize suitable bike paths
G2	Allow the user to browse different bike paths
G3	Allow the user to see bike paths statistics
G4	Allow the user to search for paths between two locations
G5	Allow the user to see paths optimality based on other registered users' reviews
G6	Allow user to see weather information when available
G7	Allow users to register to the application using credentials
G8	Allow registered user to review published bike paths
G9	Allow registered user to confirm or reject false positives in track recording
G10	Allow registered user to let the system automatically record suitable bike paths
G11	Allow registered user to manually create suitable bike paths
G12	Allow registered user to publish bike paths
G13	Allow the store of bike paths data for registered users
G14	Allow registered user to add publishable information about bike paths

Table 1.1: List of Goals

1.2. Scope

1.2.1. World Phenomena

WP	Description
WP1	A user physically moves through space using a bicycle
WP2	The physical conditions of a bike path or street change (e.g., presence of potholes, road maintenance status, or obstacles)
WP3	The meteorological conditions in the geographic area vary (e.g., temperature, wind speed)
WP4	A user intends to reach a specific destination starting from a specific origin point

Table 1.2: World Phenomena

1.2.2. Machine Phenomena

MP	Description
MP1	The system guesses that the user is biking by analyzing the detected speed
MP2	The system computes statistics related to a trip (e.g., total distance covered, average speed, performance metrics)
MP3	The system identifies potential anomalies (e.g., potholes) by processing raw data from the device sensors (accelerometer and gyroscope)
MP4	The system computes the score of a path based on its status and its effectiveness in reaching the destination
MP5	The system merges path information incoming from multiple users, evaluating data freshness and the number of confirmations received
MP6	The system stores trip history and path information in the database for future retrieval

Table 1.3: Machine Phenomena

1.2.3. Shared Phenomena

World-Controlled Phenomena

SP_WC	Description
SP_WC1	The user sends a command to start or stop recording a trip
SP_WC2	The mobile device sensors (GPS, accelerometer, gyroscope) transmit position and movement data to the system
SP_WC3	The user manually inserts data regarding a street (e.g., street name, status)
SP_WC4	The user confirms or corrects information automatically detected by the system (e.g., confirming a pothole)
SP_WC5	The user inputs search parameters for a path (origin and destination)
SP_WC6	The external weather service transmits current condition data (e.g., temperature, wind) to the system

Table 1.4: Shared Phenomena – World-Controlled

Machine-Controlled Phenomena

SP_MC	Description
SP_MC1	The system visualizes available bike paths between two points on a map, highlighting those with better scores
SP_MC2	The system displays trip statistics to the user (speed, distance, etc.)
SP_MC3	The system presents a confirmation request to the user regarding automatically detected information

Table 1.5: Shared Phenomena – Machine-Controlled

1.3. Definitions, Acronyms, Abbreviations

- BBP: Bike Buddy Project
- RASD: Requirements Analysis and Specification Document
- WP: World Phenomenon
- MP: Machine Phenomenon
- SP_WC: Shared Phenomenon - World Controlled
- SP_MC: Shared Phenomenon - Machine Controlled

1.4. Reference Documents

- Specification document assignment
- IEEE standard documentation for RASD
- Materials for the Software Engineering 2 course made available on WeBeep

1.4.1. Document Structure

This document is organized into four main chapters, as outlined below:

- **Chapter 1 - Introduction:** Provides an overview of the BBP (Best Bike Paths) system, defining its goals, the scope of the project, and the terminology used. It also lists the reference documents and the domain phenomena (World, Machine, and Shared).
- **Chapter 2 - Overall Description:** Offers a high-level perspective on the product. It includes the product perspective through scenarios, the domain model represented by a Class Diagram, and State Diagrams to describe the trip recording logic. It also lists user characteristics and the domain assumptions.

- **Chapter 3 - Specific Requirements:** This is the core of the document. It details the functional requirements through Use Case diagrams and descriptions. It also defines the non-functional requirements (performance, safety, security, etc.) and the external interface requirements (User, Hardware, and Software interfaces).
- **Chapter 4 - Formal Analysis using Alloy:** Presents the formal modeling of the system using the Alloy language. It includes the signatures, facts, and assertions used to verify the consistency of the domain model and the core logic of trip registration.

2 Overall Description

2.1. Product perspective

2.1.1. Scenarios

Scenario 1: Path Visualization
Alice is a student who wants to bike from her university to a new gym she has never visited before. She opens the Best Bike Paths (BBP) application and inputs her current location as the origin and the gym's address as the destination. The system calculates available routes and displays them on a map. BBP visualizes the paths based on a calculated score , which considers both the distance and the quality of the road surface. Alice notices that the shortest path is marked with a lower score because parts of it are tagged as "requires maintenance." She decides to select a slightly longer but "optimal" path to ensure a smoother ride. She mounts her phone on the bike handlebar and follows the suggested route.

Scenario 2: Automated Tracking and Pothole Detection
Bob is an avid cyclist who tracks his daily training sessions. He gets on his bike and starts pedaling without manually interacting with the application. BBP detects his movement speed and guesses that he is biking, automatically initiating the data acquisition process. As Bob cycles through a park, he accidentally hits a deep pothole. The application collects data from the smartphone's accelerometer and gyroscope, registering the sudden movement as a potential anomaly. At the end of the ride, Bob checks his smartphone. The system displays a summary of the trip and asks for verification regarding the anomaly detected in the park. Bob confirms that there was indeed a pothole, and this information is stored to update the community map.

Scenario 3: Scenario 3: Manual Reporting of Road Status
Charlie is cycling home when he encounters a newly repaved bike lane that was previously full of cracks. He stops safely, opens the BBP application, and chooses to insert information in manual mode. He selects the specific street segment on the map and updates its status to "Optimal." Since no other conflicting data has been entered recently, BBP immediately considers this path as optimal. Later that day, other users planning a route through that area see the updated status and choose that path for their commute.

Scenario 4: Scenario 4: Analyzing Performance and Weather

Diana has just finished a long weekend ride. She opens BBP to review her activity history. The system presents a list of her stored trips. She selects the most recent one to view the details. BBP displays her total distance covered, average speed, and other performance metrics. Additionally, since the system successfully retrieved data from the external meteorological service during her ride, the trip log is enriched with weather information, showing that she maintained a high average speed despite strong headwinds and low temperatures.

2.1.2. Class diagram

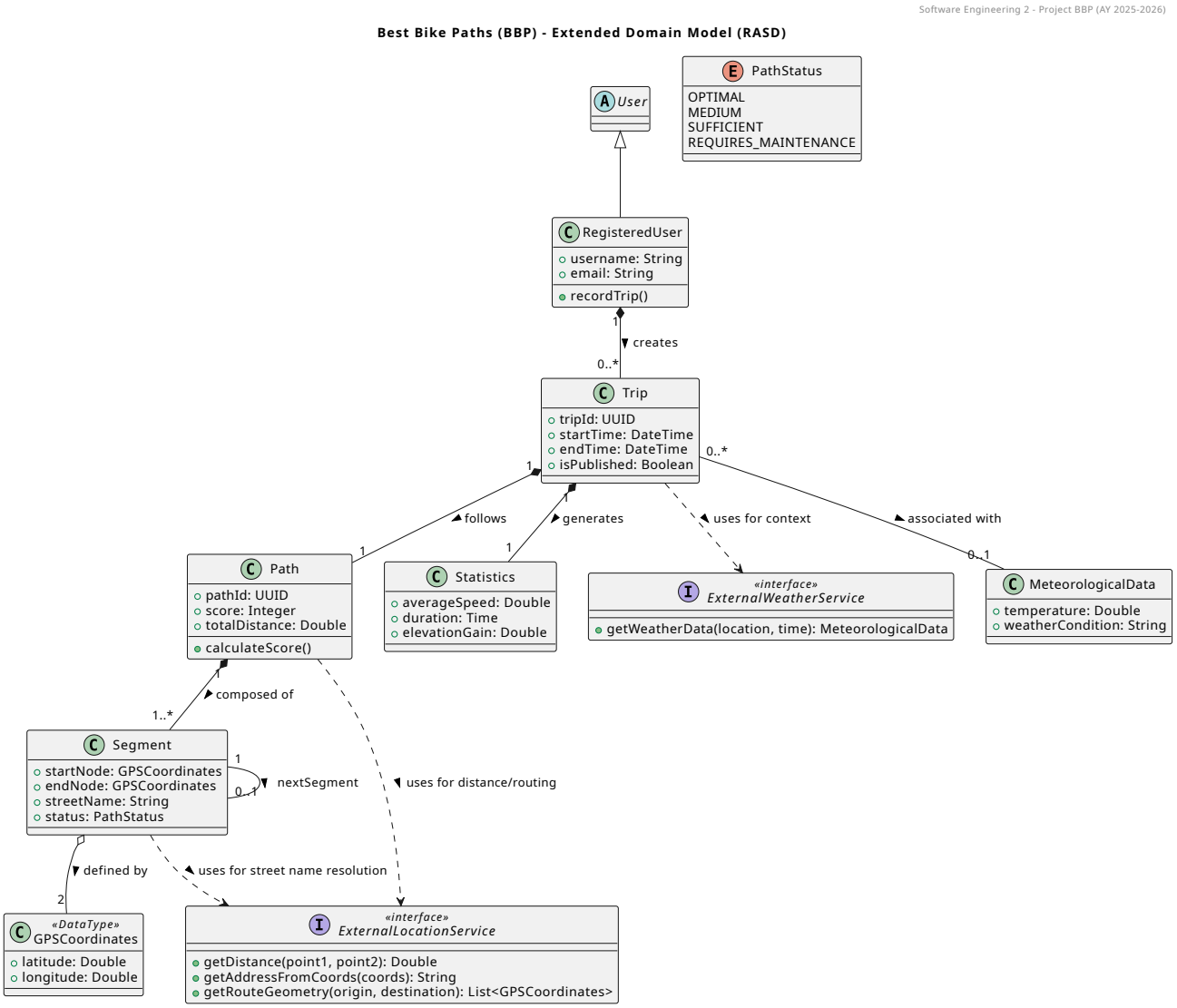


Figure 2.1: Class diagram representing the main entities and their relationships in the BBP system.

2.1.3. State diagrams

Login and Registration This diagram depicts the user authentication flow. The application defaults to Guest Mode, which allows limited access to map visualization and routing. To perform active tasks like trip recording or data reporting, the user must enter an `Authenticated Session`. The model details the transitions for logging in, signing up, and handling session termination.

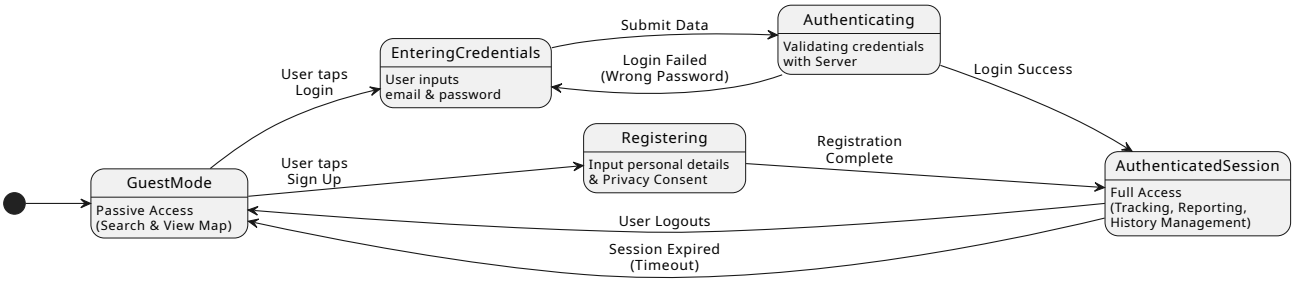


Figure 2.2: State diagram of the trip recording process.

Automated trip recording The state diagram depicts the trip lifecycle, originating in the `MonitoringSpeed` state where the system samples GPS data. The transition to `Recording` occurs either manually or automatically when a speed threshold is exceeded, actively collecting sensor data in `AcquiringData` or temporarily shifting to `Paused` when stationary. The process concludes in `AwaitingAction`, where the user reviews statistics and confirms anomalies before the trip is finally `Saved` or `Discarded`.

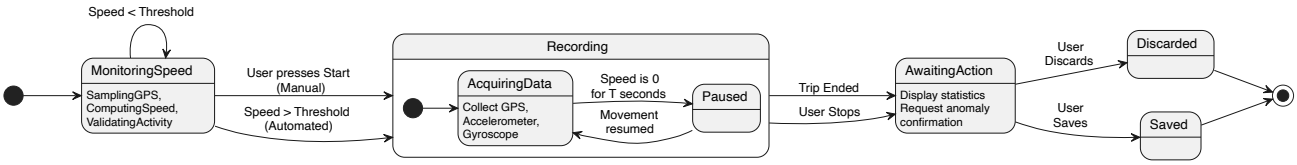


Figure 2.3: State diagram of the trip recording process.

Manual trip recording This diagram models the workflow for manually logging a trip after it has been completed ("offline reporting"). Since no real-time GPS tracking is involved, the user first provides general metadata (Origin, Destination, Duration). Subsequently, the user reconstructs the route by inputting the sequence of streets traversed. The system then allows the user to annotate specific segments with details such as potholes or road status before finalizing the submission.

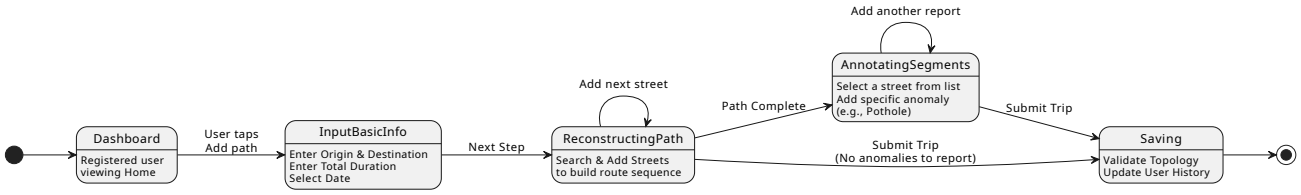


Figure 2.4: State diagram of the manual trip registration process.

Path data aggregation This diagram illustrates the backend logic for updating path attributes when new user data is submitted. The system first matches the recorded GPS trace against existing road segments in the database. It then compares the new report with historical data. If the new data confirms the existing status, the confidence score is boosted. If a conflict arises (e.g., a user reports a pothole on a previously "Optimal" path), the system applies a weighting algorithm based on data freshness to resolve the discrepancy and update the public score.

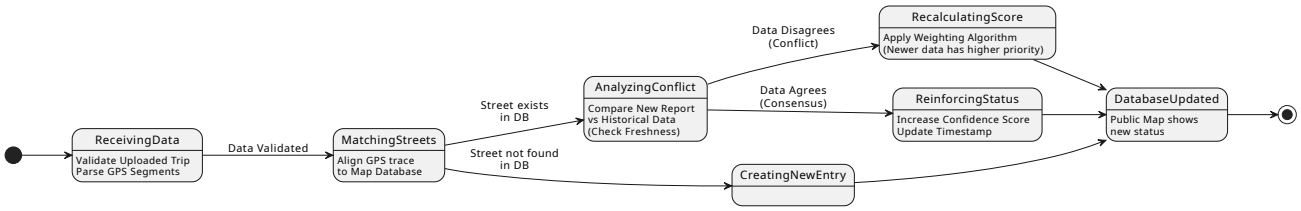


Figure 2.5: State diagram of the path data aggregation process.

Anomaly Detection and Validation This diagram illustrates the lifecycle of a road anomaly (e.g., a pothole) from detection to publication. Since sensor data can generate false positives, the system implements a validation step. Potential anomalies are first flagged internally as "Candidates" and presented to the user at the end of the trip for confirmation before being stored as valid data.

2.2. Product functions

User Registration and Authentication: Allows new users to create an account to access personalized features such as trip recording and data contribution. It handles secure login procedures and manages session states, ensuring that only authenticated users can modify the system's data.

Guest Access Management: Provides limited access to unregistered users (guests), allowing them to browse the map and search for bike paths without logging in. This function ensures that the public utility aspect of the system—viewing shared path information—is available to everyone.

Account and Profile Management: Enables registered users to manage their personal information and preferences. This includes viewing their stored data and managing their privacy settings regarding the publishable information they contribute to the community.

Path Search and Visualization: Allows users (both registered and guests) to specify an origin and a destination to retrieve available bike paths. It renders these paths on an interactive map, highlighting the most effective routes based on the system's scoring algorithm.

Trip Recording (Automated and Manual): Manages the acquisition of trip data. It supports a manual start/stop mechanism and an automated mode that "guesses" when the user is biking based on speed. It collects GPS coordinates for the route and sensor data (accelerometer/gyroscope) for anomaly detection.

Path Status Reporting: Provides an interface for users to manually insert data about street segments, such as maintenance status (e.g., "optimal," "requires maintenance") or specific obstacles. It also handles the user validation step required to confirm automatically detected anomalies like potholes.

Trip History and Statistics: Stores the details of completed trips for registered users, creating a personal log of cycling activities. It computes and displays key performance metrics for each trip, such as total distance covered and average speed.

Meteorological Data Integration: Connects to external weather services to retrieve environmental conditions (temperature, wind speed) for the specific time and location of a trip. It enriches the trip history with this context to help users analyze their performance relative to the weather.

Path Scoring and Ranking: Implements the logic to calculate a "score" for each path segment. This calculation considers the physical status of the road and its effectiveness in connecting the origin to the destination, ensuring the best paths are suggested first.

Information Merging and Data Consistency: Aggregates data received from multiple users regarding the same path segment. It applies specific rules to resolve conflicts—prioritizing data freshness and the number of confirming reports—to determine the current, most accurate status of a bike path.

2.3. User charateristic

The user population of the Best Bike Paths (BBP) system consists primarily of individuals who use bicycles for commuting, leisure, or sport. The user base is heterogeneous regarding age and technical skills, assuming only a basic familiarity with smartphone applications.

We identify two main categories of users:

- **Registered Users**: This category represents the core user base of the application.
  - *Goals*: They use the system to track their cycling activities, store trip history, monitor performance statistics (e.g., speed, distance), and contribute to the community by reporting road conditions.
  - *Capabilities*: They have access to all system features, including data acquisition (Manual and Automated modes) and data visualization.
  - *Context*: They are willing to share their location data and sensor readings to assist the community and maintain a personal log of their rides.
- **Unregistered Users (Guests)**: This category includes occasional users or those concerned with privacy who do not wish to create an account.
  - *Goals*: Their primary goal is to use the "passive" features of the system, specifically searching for bike paths and visualizing the best routes between an origin and a destination.
  - *Capabilities*: Their access is limited to the browsing and routing functionalities; they cannot record trips, save history, or contribute data to the system.

2.4. Assumptions, dependencies and constraints

2.4.1. Domain assumptions

- **GPS Accuracy:** It is assumed that the GPS sensors on users' mobile devices provide a level of accuracy sufficient to identify specific street segments, typically within a range of 5–10 meters.
- **Internet Connectivity:** Users are assumed to have access to a stable internet connection (WiFi or mobile data) at the end of their trips to allow the synchronization of recorded data with the BBP servers.
- **Map Service Reliability:** External map providers (e.g., Google Maps, OpenStreetMap) are assumed to provide up-to-date and accurate information regarding road networks, street names, and cycling-compatible paths.
- **User Honesty and Reliability:** It is assumed that registered users provide truthful information when manually inserting path status or confirming automated detections of obstacles like potholes.
- **Availability of External APIs:** External services for meteorological data and geocoding are assumed to be operational and reachable via standard protocols when the system needs

to enrich trip data.

- **Device Sensor Functionality:** For automated data acquisition, it is assumed that the users' smartphones are equipped with functioning accelerometers and gyroscopes capable of detecting significant movement anomalies.
- **Path Decomposition:** Every bike path can be logically represented as a continuous sequence of segments, where each segment corresponds to a physical portion of the road network.

3 Specific Requirements

3.1. Functional Requirements

3.1.1. Requirements

ID	Description
R1	The system shall allow new users to register an account
R2	The system shall allow user not registered to access without an account if they don't want to register
R3	The system shall allow registered users to log in to the application
R4	The system shall allow registered users to view and update their personal profile information
R5	The system shall allow users to log out of their current session
R6	The system shall allow registered users to record their cycling trips
R7	The system shall store recorded trips to maintain a history of the user's cycling activities
R8	The system shall compute and display total distance covered for each trip
R9	The system shall compute and display average speed for each trip
R10	The system shall compute and display total time spent speed for each trip
R11	The system shall compute and display elevation gain for each trip
R12	The system shall compute and display calories spent for each trip
R13	The system shall compute and display additional performance metrics for the trips
R14	The system shall retrieve meteorological data (weather conditions, temperature, wind speed) from an external service and enrich the trip data when available
R15	The system shall allow registered users to insert information about bike paths
R16	The system shall allow registered users to make publishable information about bike paths

Continue in the next page...

ID	Description
R17	The system shall allow registered users to add info about the status of a bike path
R18	The system shall allow registered users to add the presence of relevant obstacles on the bike path
R19	The system shall recognise if the users are cycling on an exists bike path
R20	The system shall recognise if the users are cycling on a path where cars are rare and speed limits under 50kmh
R21	The system shall calculate the distance of a path
R22	The system shall allow registered users to manually insert bike path data by specifying the names of the streets included in the path
R23	The system shall allow registered users to manually specify the status of the street
R24	The system shall allow users to manually report the presence of obstacles on the street
R25	The system shall automatically infer that the registered user is cycling based on their speed
R26	The system shall collect GPS data to reconstruct the path followed by the user during the ride
R27	The system shall acquire data from the mobile device's accelerometer and gyroscope to detect basic metrics
R28	The system shall analyze sensor data to identify potential path irregularities
R29	The system shall require the user to confirm or correct the automatically acquired data
R30	The system shall allow registered users to mark their inserted data as "publishable" to make it available to the community
R31	The system shall merge information about the same path from multiple users
R32	The system shall prioritize data based on timeframe of execution and the number of confirmations to determine the current status
R33	The system shall allow any user to search for bike paths by specifying an origin and a destination
R34	The system shall find for its data bike paths where an origin and a destination are specified by any users
R35	The system shall visualize the bike paths connecting the specified origin and destination on a map
R36	The system shall compute a score for each path based on its status and its effectiveness in reaching the destination from the origin

Continue in the next page...

ID	Description
R37	If multiple paths exist, from the user's request the system shall display them sorted by their calculated score
R38	The system shall collect paths data and keep it saved
R39	The system shall visualize paths data
R40	The system shall filter paths through various attributes
R41	The system shall allow the user to search through public paths
R42	The system shall be able to comunicate with a weather API
R43	The system shall be able to comunicate with an API for finding cycling routes
R44	The system shall be able to comunicate with an API for calculate the distance between 2 coordinates
R45	The system shall allow registered users to manually insert the duration of the trip
R46	The system shall distinguish between declared public trip and private ones
R47	The system shall allow registered users to visualize a summary of the trip

Table 3.1: Requirements List

3.1.2. Use Cases Diagrams

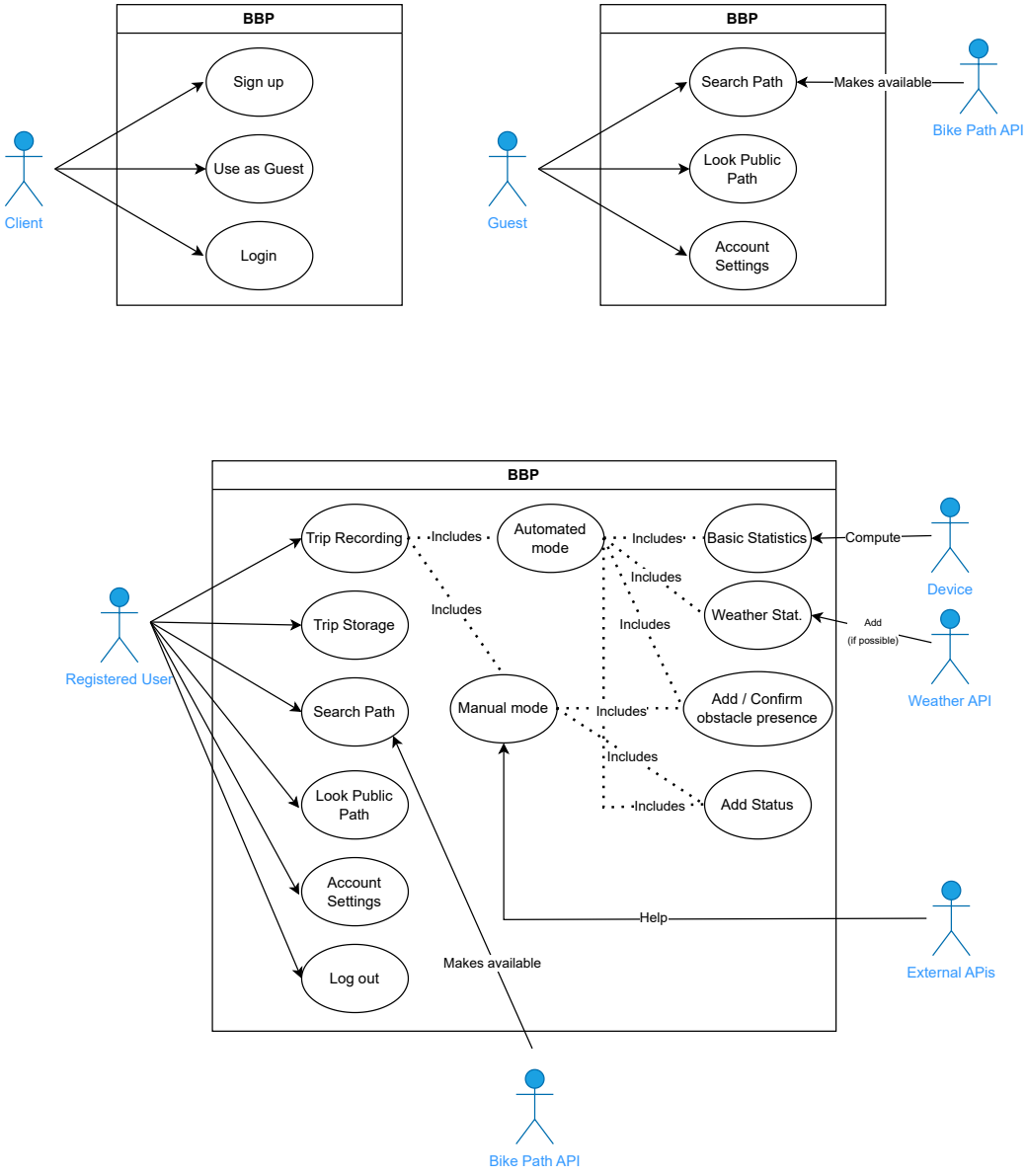


Figure 3.1: Use Cases

Use Case Diagram Description

The provided Use Case diagrams illustrate the functional requirements and actor interactions within the Best Bike Paths (BBP) system. The diagrams are structured to differentiate between the levels of access granted to users:

- Authentication and Access: The initial view depicts the generic Client actor, representing an unauthenticated user who interacts with the system's entry points to Sign Up, Log In, or proceed as a Guest
- Guest Functionality: The Guest actor is afforded limited privileges, primarily focused on information retrieval. Their interactions utilize the Bike Path API to "Search Path" and "Look Public Path," allowing users to visualize routes without contributing new data
- Registered User Functionality: The detailed main diagram focuses on the Registered User, who has access to the full suite of BBP features, including "Trip Storage" and "Trip Recording." A key component is the "Trip Recording" use case, which supports two distinct data ingestion methods:
  - Manual Mode: Allows the user to input street names and status directly
  - Automated Mode: Integrates with secondary actors—specifically the Device (accelerometers/gyroscopes) and the Weather API—to automatically compute basic statistics, retrieve meteorological data, and detect potential obstacles or status updates during a ride

3.1.3. Use Cases

In this section, they are explained and represented the main identified use cases. For each Use Case, the documentation follows a two-step structure:

- 1. Use Case Table: A structured table detailing specific fields
- 2. Diagram: A Sequence Diagram visualizing the steps described in the table

UC1. Sign up to BBP

Actor	Unregistered User, BBP
Entry conditions	The User is not already registered in BBP and has decided to sign up for the app
Event Flow	1. User click to register the "Create Account" button 2. BBP show the registration form 3. User fills in the fields by entering his email address and password 4. BBP used the data to create a new account 5. If it's all correct the BBP show the app as registered user
Exit condition	BBP allows the User to access the application
Exceptions	1. The email address is not valid (Possible cases could be: email already link to another account or email does not exist), in this case BBP return the Sign Up form included a banner where it's shown "The email is not valid" 2. The password is not valid, in this case BBP return the Sign Up form included a banner where it's shown "The password is not valid"

Table 3.2: Sign up to BBP

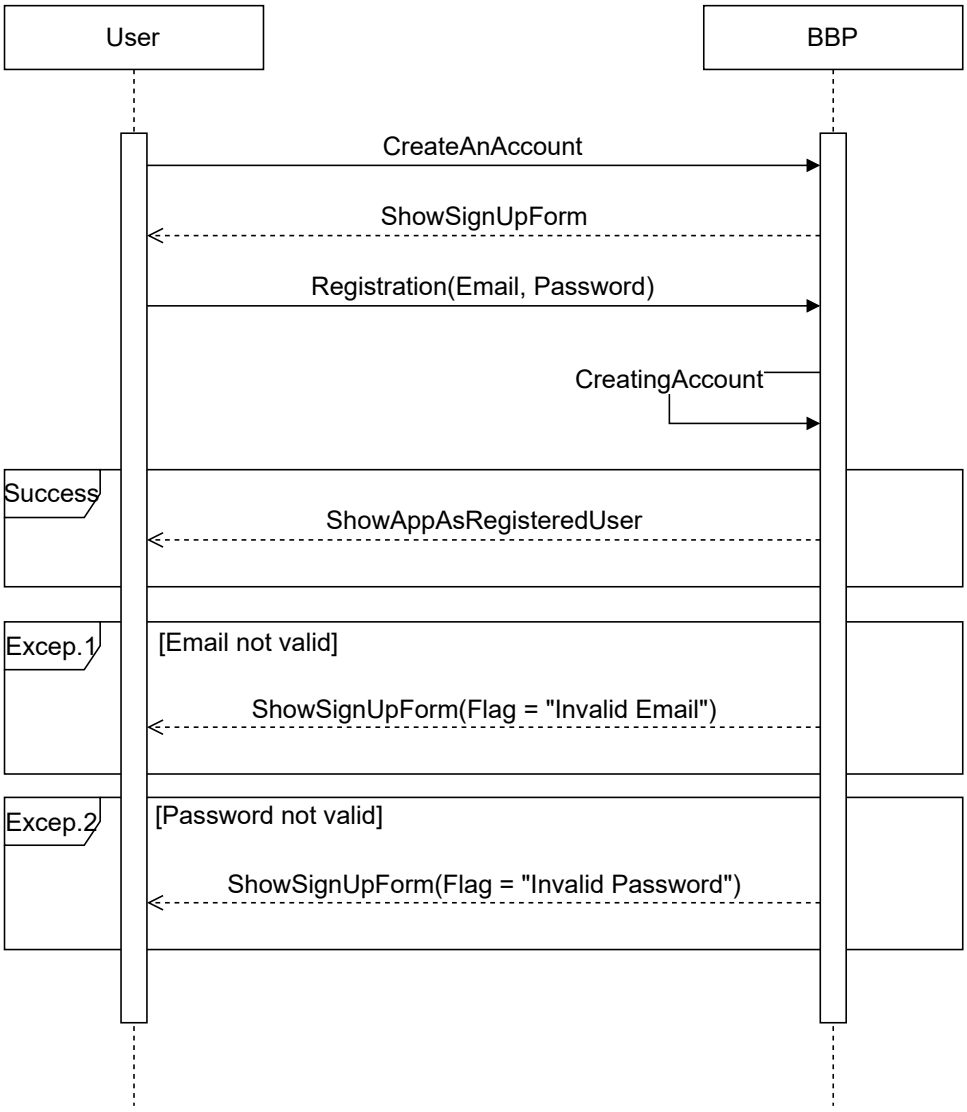


Figure 3.2: Sign up to BBP

UC2. Login to BBP

Actor	User, BBP
Entry conditions	The User is already registered in BBP and has decided to login
Event Flow	1. User fill the Login form with email and password 2. BBP checks the credentials 3. If they are correct, BBP shows the app as registered user
Exit condition	BBP allows the User to access the application
Exceptions	1. The email address is not valid, in this case BBP return the form included a banner where it's shown "The email is not valid" 2. The password is not valid, in this case BBP return the form included a banner where it's shown "The password is not valid" 3. The email is not registered to any account, in this case BBP return the form included a banner where it's shown "The email is not link to any account"

Table 3.3: Login to BBP

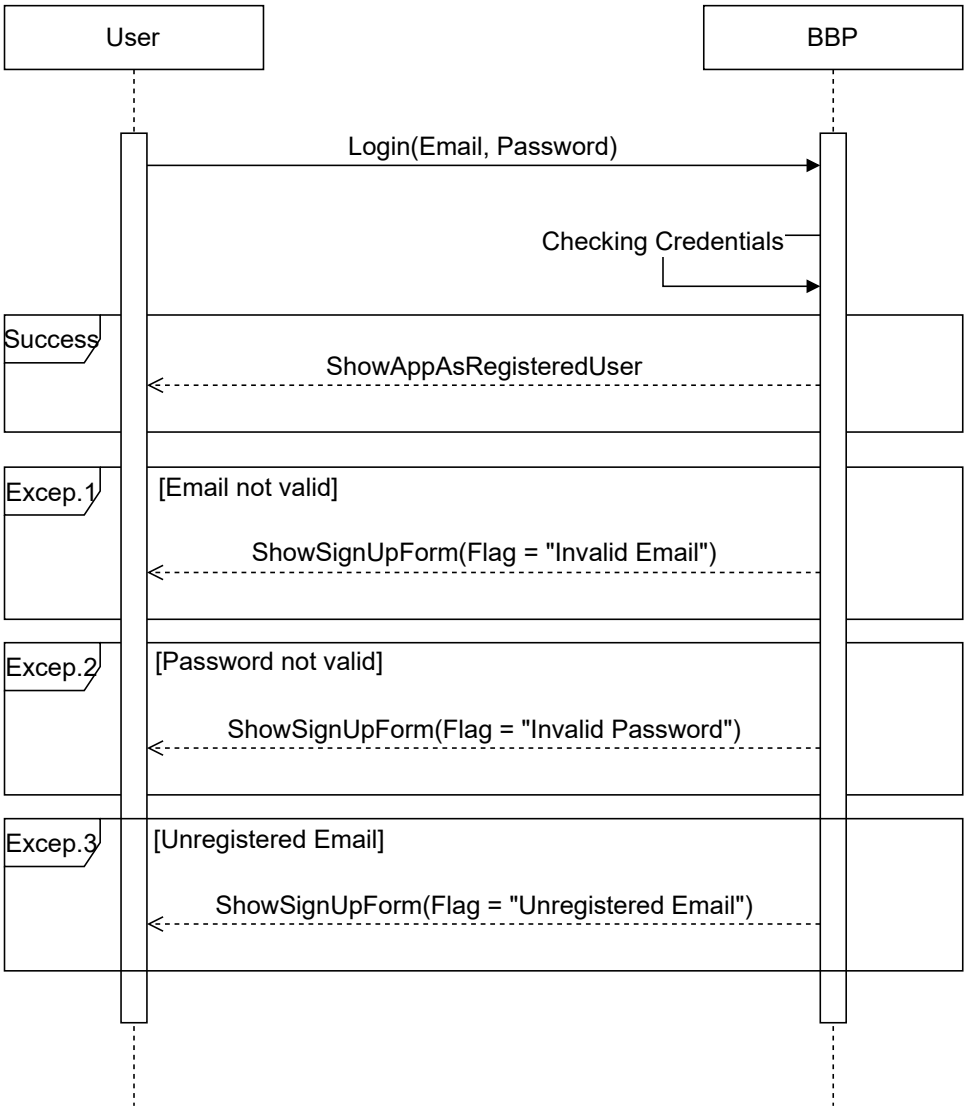


Figure 3.3: Login to BBP

UC3. Logout to BBP

Actor	Registered User, BBP
Entry conditions	User want to logout from the app
Event Flow	1. Registered User press the "Logout" button 2. BBP shows a banner asking if the User want to logout from his account 3. Registered User answers "Yes" 4. If it's all correct, BBP logs out the user and displays the application in guest mode
Exit condition	BBP shows the application as a guest user
Exceptions	1. Could be an internal error during the logout so if happens BBP shows a banner underline the error type and asking for the logout

Table 3.4: Logout to BBP

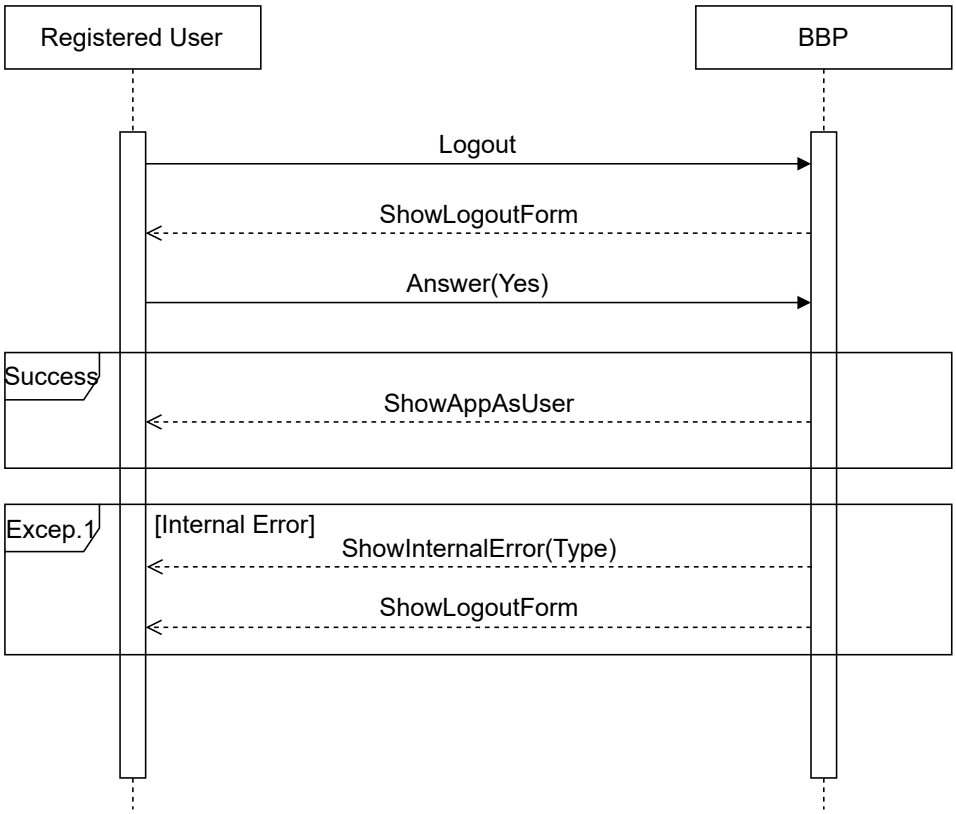


Figure 3.4: Logout to BBP

Actor	Registered User, BBP, External APIs
Entry conditions	User has dediced to start a recording in manual mode
Event Flow	1. User start a recording in manual mode 2. BBP shows a banner "Recording started" 3. BBP shows a page where user have to insert the starting point of his trip 4. User insert the coordinates of his starting point 5. If it's all right, BBP shows a confirmation banner 6. First loop : every node that user pass (a) User insert the coordinates of his next node (b) Optional : User insert the obstacles on the road (c) Optional : User insert the road status (d) BBP shows a confirmation banner (e) BBP create a segment with the last two nodes 7. User press the button "End trip" 8. BBP shows an ending banner 9. User insert the starting and finishing time 10. Second loop : every segment created (a) BBP asks the distance of a segment to API (b) API returns the distance (c) BBP used these data to fill its fields 11. BBP calculate the metrics 12. BBP shows a small summary of the activity 13. BBP save the trip in the personal storage area
Exit condition	User has finished his trip
Exceptions	1. If the node it's not valid BBP shows an error banner and reload the insertion page 2. If accur an API error BBP skip that segment and process the next one 3. If starting time is greater than finishing time BBP shows the error and reload the insertion time page

Table 3.5: Recording with Manual Mode

UC4. Recording with Manual Mode

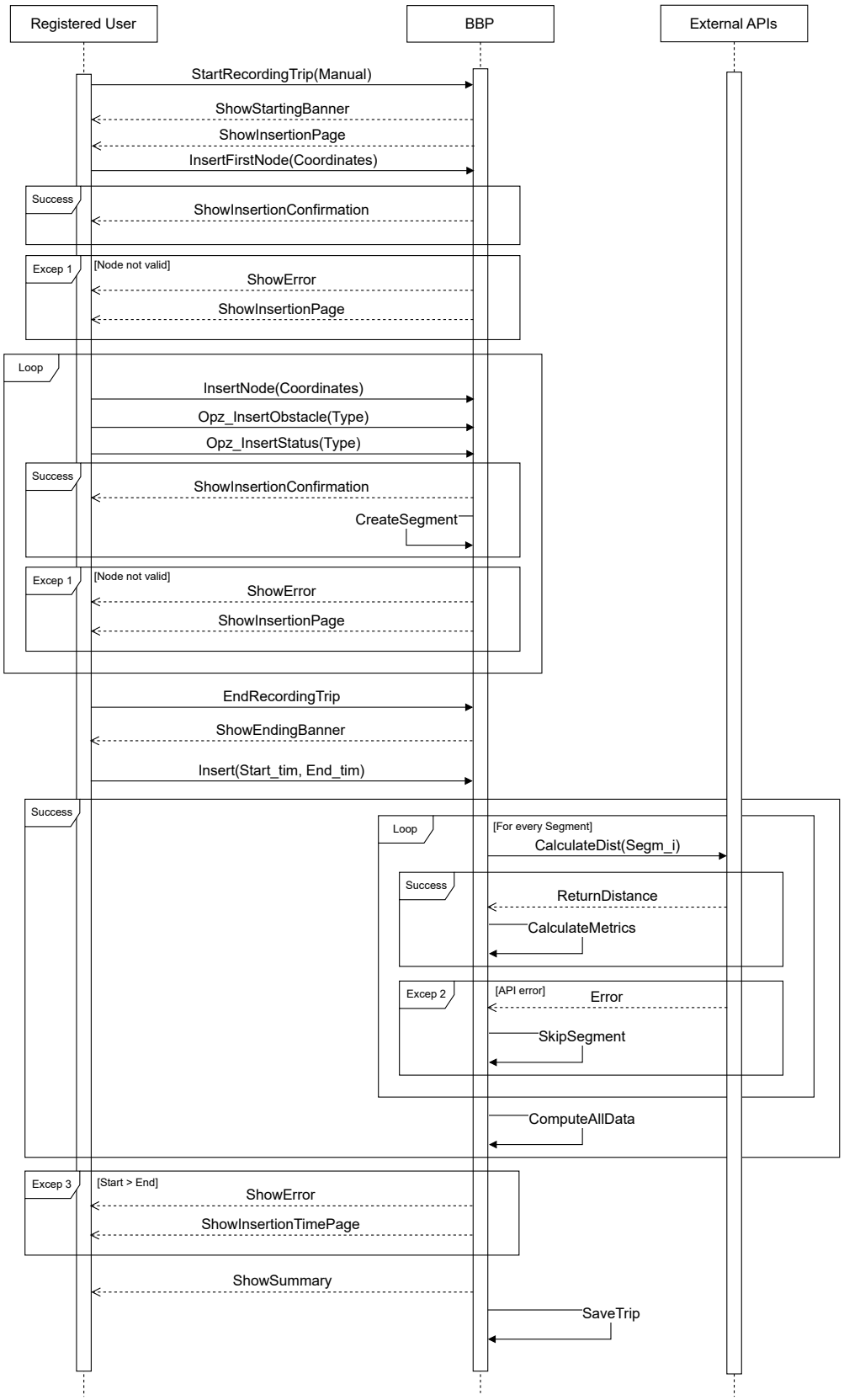


Figure 3.5: Recording with Manual Mode

UC5. Recording with Automated Mode

Precondition	Exclude all possible exceptions from the device
Actor	Registered User, BBP, Device, Weather API, External APIs
Entry conditions	User has decided to start a recording in automated mode
Event Flow 1	1. First Loop: every second (a) BBP asks the device used for metrics like postion, acceleration and gyroscope data (b) Device returns the data (c) BBP used these data to calculate if the user is biking or not 2. If the user is biking, the app show a banner asking if the User want to start a recording 3. If the User answers "Yes", the app start a recording in automated mode 4. BBP shows a banner "Recording started" 5. Second loop : every second (a) BBP asks the device used for metrics like postion, acceleration and gyroscope data (b) Device returns the data (c) BBP used these data to create a segment (Only from the second iteration) (d) BBP ask to External APIs to calculate the distance of the segment (Only from the second iteration) (e) If it's all right, External APIs returns the distance (Only from the second iteration) (f) BBP used these data to calculate the metrics (Only from the second iteration)

continue...

...continue	
Event Flow 2	1. Third loop : every 5 minutes (a) BBP asks the weather condition to API (b) API returns the data (c) BBP used these data to fill its fields 2. User press the button "End trip" 3. BBP compute all data 4. BBP shows a small summary of the activity 5. BBP save the trip in the personal storage area
Exit condition	User has finished his trip
Exceptions	1. External APIs could produce an error after the request for distance, if it's the case the BBP skip that segment and process the next one 2. Weather API could produce an error and return nothing to BBP, if it's the case BBP will fill its fields with "Data not available"

Table 3.6: Recording with Automated Mode

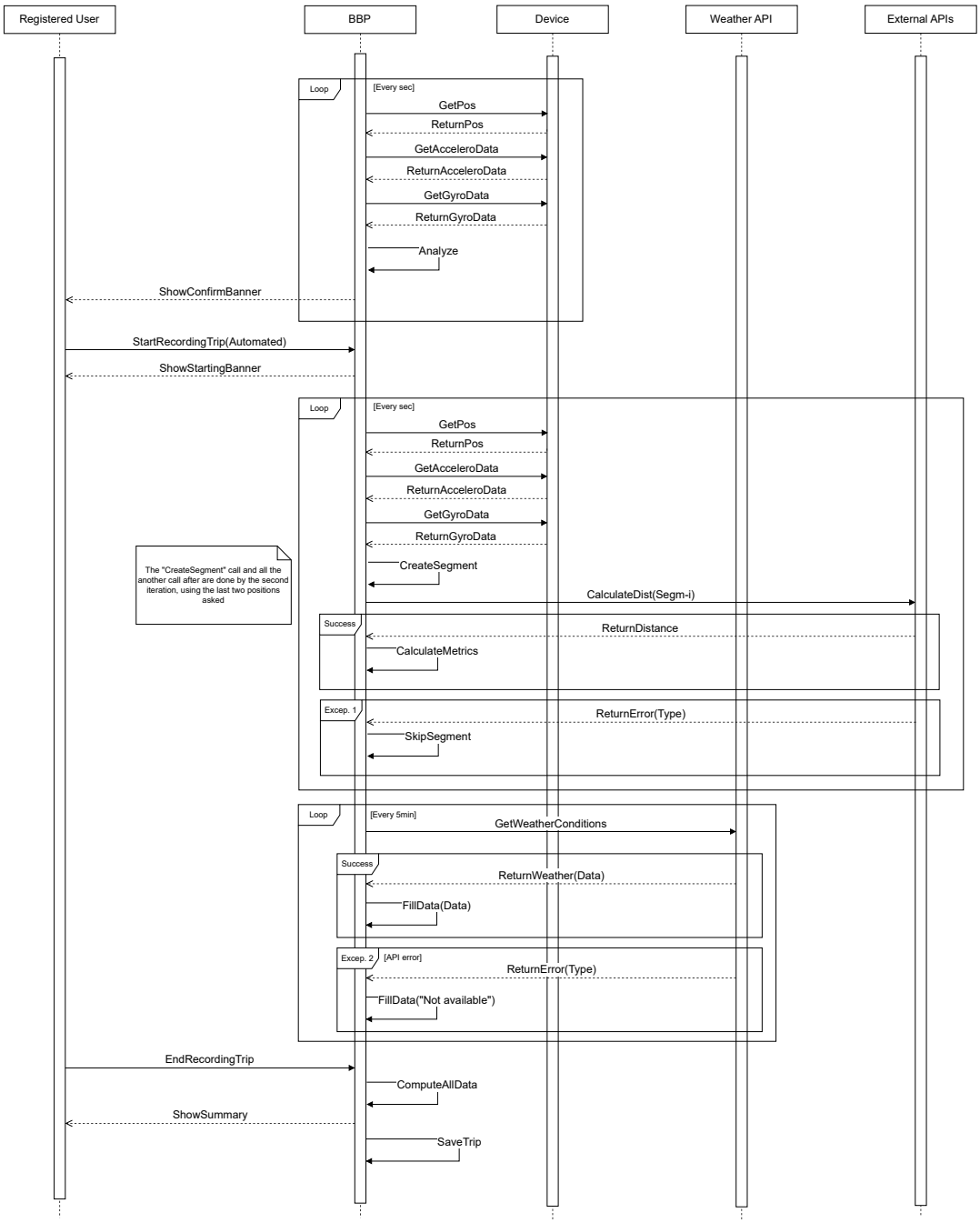


Figure 3.6: Recording with Automated Mode

UC6. Personal Trip Storage

Actor	Registered User, BBP
Entry conditions	User want to view his completed and saved trips
Event Flow	1. User clicks the section about personal trip storage 2. BBP shows the trips saved and taken by the User 3. User selects a specific trip (currently private) to view its report and make it public 4. BBP returns a screen with the trip report 5. User clicks the button to make the trip public 6. If it's all right, BBP updates the page with the now public trip and adds it to the public search for data merge 7. User selects another trip (currently public) to view its report and make it private 8. BBP returns a screen with the trip report 9. User clicks the button to make the trip private 10. If it's all right, BBP updates the page with the now private trip and delete it to the public search for data merge
Exit condition	User has finished to view his past trips
Exceptions	1. During the change from private to public and vice versa, an internal problem may occur. In this case, BBP returns a banner with the error type and reloads the trips page without any changes having occurred

Table 3.7: Personal Trip Storage

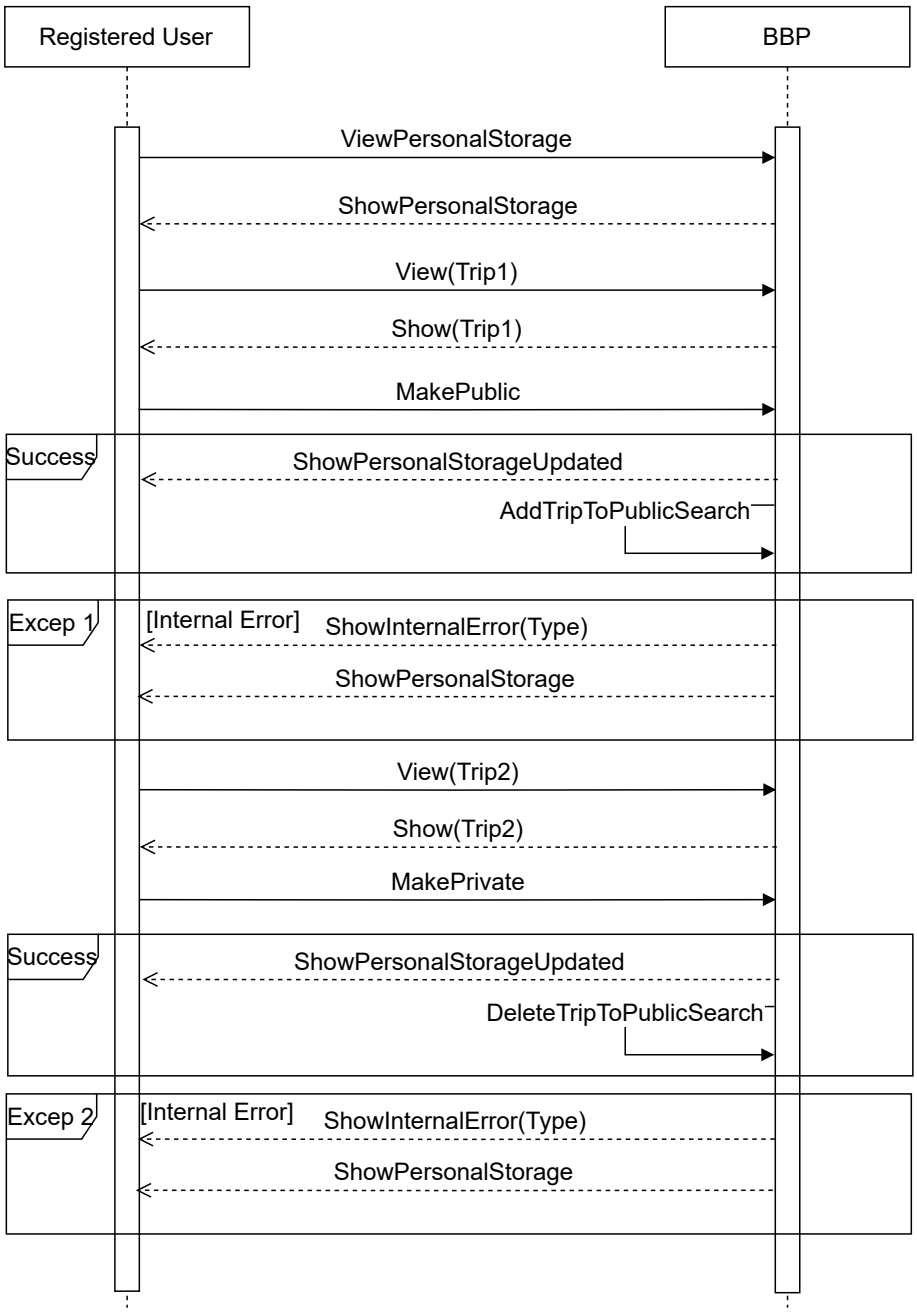


Figure 3.7: Personal Trip Storage

UC7. Search Path

Actor	Registered User, BBP
Entry conditions	User want to search a path from a certain starting point to an ending point
Event Flow	1. User requests a path search through 2 selected points 2. BBP shows a banner with the search loading 3. BBP find all paths that pass through those 2 points 4. If it's all right, BBP returns the paths found
Exit condition	User can consult the search results to see the most suitable path
Exceptions	1. If the start or end points do not exist, BBP returns a banner with the words "Points not valid" and reloads the empty search page ready for new data entry 2. If no paths exist, BBP returns a banner with the words "No paths found" and reloads the empty search page ready for new data entry

Table 3.8: Search Path

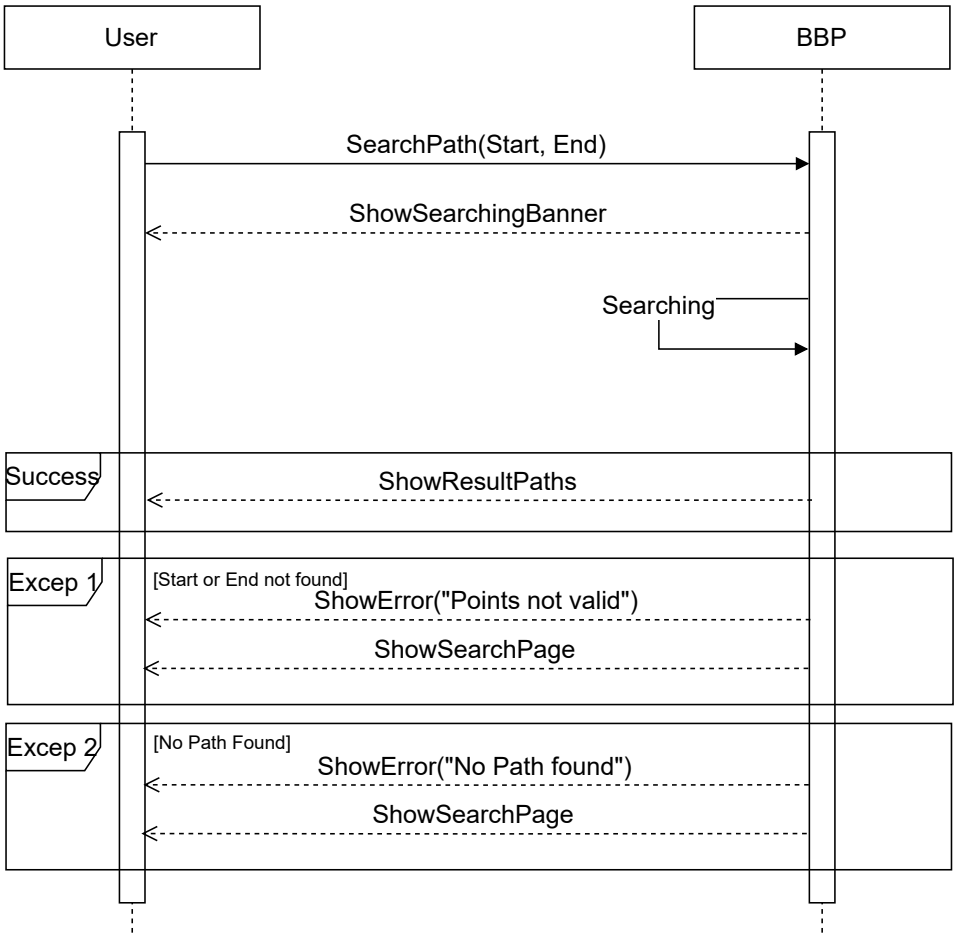


Figure 3.8: Search Path

UC8. Look Publish Path

Actor	Registered User, BBP
Entry conditions	User wants to see the trips published by other users
Event Flow	1. User requests to view the routes published in his area 2. BBP search between the pubblic paths 3. If it's all right, BBP returns the results 4. User selects a path 5. BBP return the path's summary
Exit condition	User has finished to visualize other public paths
Exceptions	1. During the selection of a path, an internal problem may occur. In this case, BBP returns a banner with the error type and reloads the public path page 2. If the area does not exist, BBP returns a banner with the words "Area not valid" and loads the empty search page ready for new data entry 3. If no public path exist, BBP returns a banner with the words "No public path found" and loads the empty search page ready for new data entry

Table 3.9: Look Publish Path

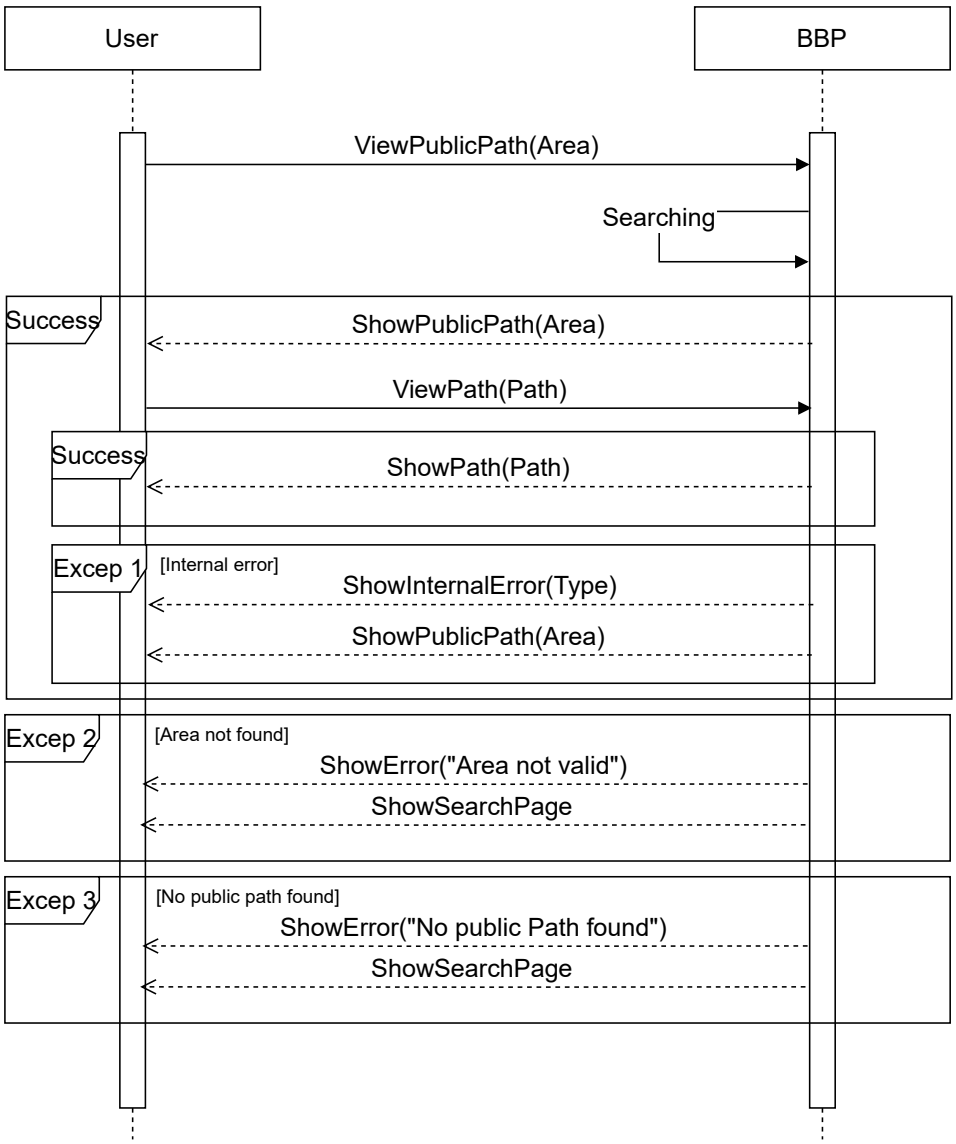


Figure 3.9: Look Publish Path

UC9. Obstacles detection

Precondition	Exclude all possible exceptions from the device
Actor	Registered User, BBP, Device, External APIs
Entry conditions	During a trip recording in Automated Mode BBP detected an obstacle
Event Flow	1. User start a recording in automated mode 2. BBP shows a banner "Recording started" 3. First loop : every second (a) BBP asks the device used for metrics like postion, acceleration and gyroscope data (b) Device returns the data (c) BBP used these data to create a segment (Only from the second iteration) (d) BBP ask to External APIs to calculate the distance of the segment (Only from the second iteration) (e) If it's all right, External APIs returns the distance (Only from the second iteration) (f) BBP used these data to calculate the metrics (Only from the second iteration) (g) If BBP detect an obstacle, it save this information (Only from the second iteration) 4. User press the button "End trip" 5. Second loop : every obstacle detected (a) BBP shows a form about the presence of obstacle in a specific point (b) User answers Yes or No (c) BBP used this data to update the obstacles presence 6. BBP compute all data 7. BBP shows a small summary of the activity 8. BBP save the trip in the personal storage area
Exit condition	Trip is over
Exceptions	1. External APIs could produce an error after the request for distance, if it's the case the BBP skip that segment and process the next one

Table 3.10: Obstacles detection

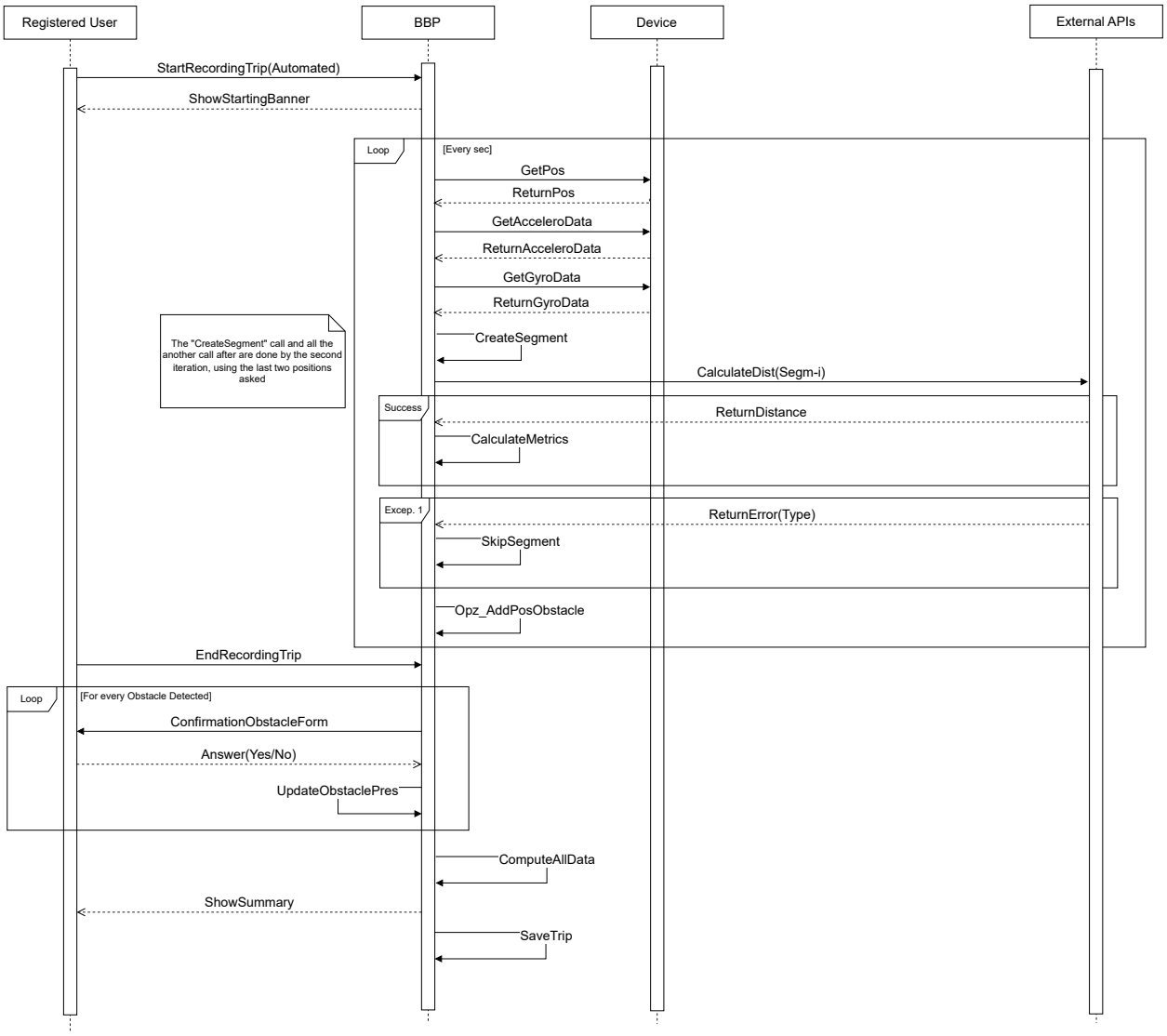


Figure 3.10: Obstacles detection

3.1.4. Mapping on Goals

G1 : Recognize suitable bike paths

- **R19:** The system shall recognise if the users are cycling on an exists bike path
- **R20:** The system shall recognise if the users are cycling on a path where cars are rare and speed limits under 50kmh
- **R28:** The system shall analyze sensor data to identify potential path irregularities

G2 : Allow the user to browse different bike paths

- **R39:** The system shall visualize paths data
- **R40:** The system shall filter paths through various attributes
- **R41:** The system shall allow the user to search through public paths

G3 : Allow the user to see bike paths statistics

- **R8:** The system shall compute and display total distance covered for each trip
- **R9:** The system shall compute and display average speed for each trip
- **R10:** The system shall compute and display total time spent speed for each trip
- **R11:** The system shall compute and display elevation gain for each trip
- **R12:** The system shall compute and display calories spent for each trip
- **R13:** The system shall compute and display additional performance metrics for the trips
- **R21:** The system shall calculate the distance of a path
- **R44:** The system shall be able to comunicate with an API for calculate the distance between 2 coordinates
- **R47:** The system shall allow registered users to visualize a summary of the trip

G4 : Allow the user to search for paths between two locations

- **R33:** The system shall allow any user to search for bike paths by specifying an origin and a destination

- **R34:** The system shall find for its data bike paths where an origin and a destination are specified by any users
- **R35:** The system shall visualize the bike paths connecting the specified origin and destination on a map
- **R43:** The system shall be able to comunicate with an API for finding cycling routes

G5 : Allow the user to see paths optimality based on other registered users' reviews

- **R32:** The system shall prioritize data based on timeframe of execution and the number of confirmations to determine the current status
- **R36:** The system shall compute a score for each path based on its status and its effectiveness in reaching the destination from the origin
- **R37:** If multiple paths exist, from the user's request the system shall display them sorted by their calculated score

G6 : Allow user to see weather information when available

- **R14:** The system shall retrieve meteorological data (weather conditions, temperature, wind speed) from an external service and enrich the trip data when available
- **R42:** The system shall be able to comunicate with a weather API

G7 : Allow users to register to the application using credentials

- **R1:** The system shall allow new users to register an account
- **R2:** The system shall allow user not registered to access without an account if they don't want to register
- **R3:** The system shall allow registered users to log in to the application
- **R4:** The system shall allow registered users to view and update their personal profile information
- **R5:** The system shall allow users to log out of their current session

G8 : Allow registered user to review published bike paths

- **R31:** The system shall merge information about the same path from multiple users
- **R32:** The system shall prioritize data based on timeframe of execution and the number of confirmations to determine the current status

G9 : Allow registered user to confirm or reject false positives in track recording

- **R29:** The system shall require the user to confirm or correct the automatically acquired data

G10 : Allow registered user to let the system automatically record suitable bike paths

- **R6:** The system shall allow registered users to record their cycling trips
- **R25:** The system shall automatically infer that the registered user is cycling based on their speed
- **R26:** The system shall collect GPS data to reconstruct the path followed by the user during the ride
- **R27:** The system shall acquire data from the mobile device's accelerometer and gyroscope to detect basic metrics
- **R28:** The system shall analyze sensor data to identify potential path irregularities

G11 : Allow registered user to manually create suitable bike paths

- **R22:** The system shall allow registered users to manually insert bike path data by specifying the names of the streets included in the path
- **R23:** The system shall allow registered users to manually specify the status of the street
- **R24:** The system shall allow users to manually report the presence of obstacles on the street
- **R45:** The system shall allow registered users to manually insert the duration of the trip

G12 : Allow registered user to publish bike paths

- **R16:** The system shall allow registered users to make publishable information about bike paths

- **R30:** The system shall allow registered users to mark their inserted data as "publishable" to make it available to the community
- **R46:** The system shall distinguish between declared public trip and private ones

G13 : Allow the store of bike paths data for registered users

- **R6:** The system shall allow registered users to record their cycling trips
- **R7:** The system shall store recorded trips to maintain a history of the user's cycling activities
- **R31:** The system shall merge information about the same path from multiple users
- **R38:** The system shall collect paths data and keep it saved

G14 : Allow registered user to add publishable information about bike paths

- **R15:** The system shall allow registered users to insert information about bike paths
- **R17:** The system shall allow registered users to add info about the status of a bike path
- **R18:** The system shall allow registered users to add the presence of relevant obstacles on the bike path

Req.	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	G11	G12	G13	G14
R1							✓							
R2							✓							
R3							✓							
R4							✓							
R5							✓							
R6										✓			✓	
R7													✓	
R8			✓											
R9			✓											
R10			✓											
R11			✓											
R12			✓											
R13			✓											
R14						✓								
R15														✓
R16												✓		
R17														✓
R18														✓
R19	✓													
R20	✓													
R21			✓											
R22											✓			
R23											✓			
R24											✓			
R25										✓				
R26										✓				
R27										✓				
R28	✓									✓				
R29									✓					
R30												✓		

Req.	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	G11	G12	G13	G14
R31								✓					✓	
R32					✓			✓						
R33				✓										
R34				✓										
R35				✓										
R36					✓									
R37					✓									
R38													✓	
R39		✓												
R40		✓												
R41		✓												
R42						✓								
R43				✓										
R44			✓											
R45											✓			
R46												✓	✓	
R47			✓											

Table 3.11: Mapping Requirements on Goals

3.2. Performance Requirements

The system relies heavily on real-time data acquisition, geospatial processing, and user interaction through mobile devices. Therefore, specific performance constraints must be met to ensure a proper user experience and the integrity of the collected data.

- **Response Time and Latency:** The system must ensure low latency during critical user interactions to maintain usability. Specifically, for general browsing and viewing of bike paths, the system should render map visualizations and path details within 3 seconds of a user's request, assuming a standard 4G/5G connection. For the route-finding feature, the algorithm computing the path score and optimal route must complete execution within 5 seconds. Regarding the "Automated mode," the system is required to process sensor inputs (accelerometer and gyroscope) and GPS data in real-time. The lag between the physical detection of an obstacle and its internal logging on the device should be very reduced to

ensure accurate timestamping and location tagging. However, the complex merging of data from the same path can be performed asynchronously on the server side, provided that the updated status becomes available to the community within a reasonable timeframe like 5 minutes of data upload

- **Number of Concurrent Users:** As a community-driven platform managed by a biker's association, BBP must support a significant number of concurrent users, particularly during periods where the numbers of bikers could increase a lot, such as weekends and holidays. The system backend must be capable of handling at least 1 mln concurrent active sessions without performance degradation
- **Data Storage and Capacity:** The storage infrastructure must be designed to accommodate high-volume geospatial and time-series data. Since the application records continuous GPS tracks, along with high-frequency accelerometer and gyroscope readings for obstacle detection, the database will grow rapidly. The system requires a scalable storage solution capable of archiving historical trip data while keeping active path inventories and recent statistics immediately accessible. Provisioning should be made for storing several Terabytes of data over the first few years of operation, with a strategy for data retention that prioritizes the freshness of path conditions
- **Network Bandwidth and Connectivity:** The system must be resilient to variable network conditions. The application should optimize data transmission to minimize bandwidth usage, which is crucial for users with limited data plans. In "Automated mode," the system must implement a "store-and-forward" mechanism: if the user enters an area with poor or no connectivity, the application must locally buffer all GPS and sensor data. Once connectivity is restored, the system should upload the accumulated data without freezing the user interface or interrupting the ongoing recording of the trip. The transmission of external meteorological data should be lightweight, fetching updates only at necessary intervals rather than continuously streaming, to preserve bandwidth
- **Energy Efficiency:** A critical performance requirement for the system's component is battery consumption. The simultaneous usage of the GPS module, accelerometer, gyroscope, and background processing algorithms constitutes a high energy drain. The application must be optimized to ensure that running the "Automated mode" doesn't reduce a standard smartphone battery excessively fast. The optimal target should be to allow a user to track a continuous ride of at least 5 to 8 hours on a fully charged device

3.3. Design Constraints

The design and development of the Best Bike Paths (BBP) system must adhere to specific boundaries imposed by the operational environment, regulatory requirements, and the technological limitations of the end-user devices. These constraints influence the architectural decisions and the selection of technologies to ensure the system is robust, legal, and usable.

3.3.1. Standard compliance

Given the nature of the application, which involves the collection, storage, and processing of user data and geolocation tracking, the system must strictly comply with relevant data protection regulations. Specifically, since the application tracks users' movements and associate trips with registered profiles, adherence to EU's GDPR (General Data Protection Regulation) is mandatory. Moreover, the system relies on external services. Therefore, it must comply with standard geospatial data formats to ensuring interoperability with standard mapping APIs and the navigation modules. The communication between the mobile application, the backend server, and the external meteorological service must adhere to modern security standards, necessitating the use of HTTPS and TLS protocols to encrypt data in transit. Finally, the integration of weather data implies compliance with the specific API standards provided by the chosen third-party meteorological service provider, like OpenWeatherMap.

3.3.2. Hardware limitations

The "Automated mode" of the BBP application introduces significant hardware dependencies that act as constraints on the system's availability and performance. The primary limitation is the dependence on specific mobile sensors. For the application to function user's mobile device must be equipped with a high-precision GPS receiver, an accelerometer, and a gyroscope. Additionally, the continuous usage of these sensors imposes a constraint regarding battery consumption. The design must optimize sensor polling rates to prevent excessive battery drainage, which would negatively impact the user experience during long bike trips. Finally, the system is constrained by network connectivity hardware. While the application collects data locally, the features related to retrieving meteorological information, publishing path data, and merging crowd-sourced information require an active internet connection. The system design must, therefore, account for intermittent connectivity scenarios, typical of outdoor cycling routes, by implementing local caching mechanisms until a stable connection is re-established.

3.4. Software System Attributes

This section details the non-functional requirements that define the quality constraints of the Best Bike Paths system. These attributes are critical to ensuring the system is not only functional but also trustworthy, accessible, and sustainable over time.

3.4.1. Reliability

Given the nature of real-time app data acquisition and community-driven content, the reliability of the system is fundamental. The system must ensure high accuracy in its Automated Mode, specifically in the interpretation of sensor data to differentiate insignificant movements from truly obstacle. Furthermore, the data merging mechanism, which consolidates conflicting path statuses based on freshness, must be algorithmically robust. It must consistently produce a single, logical status for a path without data corruption, even when multiple users submit updates simultaneously on the same path. Finally, the system must handle the failure of the external API service gracefully, in general, the system must be fault tolerant to prevent the propagation of errors and to guarantee a continuous usability.

3.4.2. Availability

For the server-side components, the system generally targets a 99.9% uptime during daylight hours to support routing requests. Downtime for maintenance should be scheduled during hours of statistically low cycling activity to minimize disruption to the user base. A critical aspect of BBP is the tolerance regarding network connectivity, because if a biker is traveling a path with a poor or inexistent coverage the application must store the data until the connection is restored.

3.4.3. Security

Security is focused on two main aspects: Data Privacy and Information Integrity. The privacy for the system is important because the tracking of user movements, origin-destination points, and cycling habits, this geolocation data. All these data are highly sensitive. All personal data, particularly GPS logs, must be encrypted for safeness. Users' trip histories must be strictly accessible only to the account owner unless explicitly published. Instead, integrity must be ensured for data about path, score, obstacles and any other data useful for the usability of the system, they must be protected against malicious manipulation. The system should implement authentication protocols to ensure that only registered users can modify path statuses.

3.4.4. Maintainability

The architecture must be modular to facilitate future updates, error correction and scalability. Specifically, the component responsible for interfacing with mobile hardware (GPS, accelerometer, gyroscope) and the external meteorological service should be decoupled from the core business logic structure. This allows developers to update sensor calibration or adopt new hardware APIs without rewriting the trip management logic or change provider for obtaining weather information. The database schema should also be designed to be extensible, allowing for the potential addition of new metrics to the trip statistics in future versions.

3.4.5. Portability

Portability is a functional necessity for the client-side application. The BBP mobile app must be compatible with the major mobile operating systems like Android and iOS to reach a sufficiently large community of bikers.

4 | Formal Analysis Using Alloy

For readability and simplicity reasons, we decided to divide the alloy analysis in two parts: in the first one we will explain how we intended to model our application in a more detailed way, showing all the relations that keep coherent our model, in the second one we will show the verification of the merging of two paths using temporal operators. As a design choice, we chose to insert the most important objects of the model simplifying the complex ones (for example, individual nodes of streets are not modeled) and we kept what we thought was essential to comprehend the entirety of the model.

4.1. Signatures

4.1.1. Status

```
abstract sig Status{}
one sig Clear extends Status{}
one sig RoadWork extends Status{}
one sig Pothole extends Status{}
```

Listing 4.1: Status signature

An object Status is used to be associated with a street segment, therefore allowing us to better describe streets and to introduce potholes. The actual signature is abstract since we wanted to make the object expandable with more variants (we added Clear, RoadWork and Pothole as examples).

4.1.2. Segment

```
sig Segment{
    nextSegment: lone Segment,
    status: some Status
}
```

Listing 4.2: Segment signature

An object Segment is used to model street segments: every segment is connected to at most

another segment (forming what actively is a chain of streets). To have consistent segments, they are to be considered as a part of a physical street and they also model direction of said street: therefore two different segments can be the same street segment but traversed in opposite direction, this implies that a necessary condition that we need to add is that a segment cannot connect to itself. Finally, a segment has some status that can be of multiple nature (a segment can be simultaneously in RoadWork and have a Pothole to allow more precise scoring) or by default is Clear (meaning that when it's Clear, it is the only status that it can have).

4.1.3. Path

```
sig Path{
    start: one Segment,
    score: Int
}
```

Listing 4.3: Path signature

Following the definition of a Segment, a Path is simply an objects that references the first segment of an arbitrarily long chain of segments. Paths also have a score and, since paths are discovered and inserted by RegisteredUsers, it is necessary that a Trip (see Trip signature) has said Path associated to it.

4.1.4. Statistics

```
sig Statistics{}
```

Listing 4.4: Statistics signature

Statistics are present only to enrich the graphical appearance of our model and are possessed by every Trip.

4.1.5. Trip

```
sig Trip{
    path: one Path,
    allStats: one Statistics
}
```

Listing 4.5: Trip signature

A Trip is an object that has one single path and the statistics associated with the trip itself inserted by the user and retrieved from the external service. Because Trips are created by the

user, it is necessary that every Trip has exactly one Path and that one RegisteredUser has registered said Trip.

4.1.6. RegisteredUser

```
sig RegisteredUser {
    registeredTrips: set Trip,
    publishedTrips: set Trip
}
```

Listing 4.6: RegisteredUser signature

A RegisteredUser models a user with ability to record trips. We decided not to model Guests in Alloy because the functionalities we wanted to test are used only by users with an account, so the users that we used have published trips and registered trips, where the published trips are naturally a subset of the registered ones.

4.2. Facts

All the following facts ensure us that the model stays coherent and that objects are created following a correct hierarchy.

4.2.1. Paths linked to Trips

```
fact pathsLinkedToTrips{
    all p: Path|(
        some t: Trip|(
            p in t.path
        )
    )
}
```

Listing 4.7: Paths linked to Trips

In order for a Path to exist, there must be a Trip that connects to it: this is used to avoid having paths that exist by themselves.

4.2.2. Statistics linked to Trips

```
fact statisticsLinkedToTrips{
    all s: Statistics|(
        some t: Trip|(
```

```
4               s in t.allStats
5           )
6
7           // for better displaying, all statistics are different
8           and all t1,t2: Trip|(
9               t1!=t2 and
10              s = t1.allStats implies s != t2.allStats
11          )
12      )
13  }
```

Listing 4.8: Statistics linked to Trips

Like before, in order for a Statistics to exist it is mandatory that a Trip connects to it; also, in order to make the model prettier, it is forced that every statistic is different from the rest (so that every Trip has a different Statistics object).

4.2.3. Trips linked to Users

```
1  fact tripsLinkedToUsers{
2      all t: Trip|(
3          some u: RegisteredUser|(
4              t in u.registeredTrips
5          )
6      )
7  }
```

Listing 4.9: User has at least one Trip

Again, in order for a Trip to exist it must be saved by a RegisteredUser.

4.2.4. User has at least one Trip

```
1  fact userHasTrips{
2      all u: RegisteredUser|(
3          #(u.registeredTrips) > 0
4      )
5  }
```

Listing 4.10: User has at least one Trip

Like before, RegisteredUser objects that don't have trips are useless in our model, so we make sure that every user has at least one.

4.2.5. Published Trips are subset to Regisetered ones

```
fact publishedTripsIsSubset{
    all u: RegisteredUser|(
        publishedTrips[u] in registeredTrips[u]
    )
}
```

Listing 4.11: Published trips are subset to Registered ones

Since one of the goals of the application is to allow users to save their trips and RegisteredUsers are allowed to also publish them, it is important to enforce that publishedTrips are in fact a subset of registeredTrips (since a registeredTrip can be published or can stay private).

4.2.6. Start segments don't have backlinks

```
fact startsAreHeads {
    all p: Path |(
        no s: Segment |(
            s in p.start.^nextSegment
            and s.nextSegment = p.start
        )
    )
}
```

Listing 4.12: Start segments don't have backlinks

This fact ensures that there's no segment that is both in a path and comes before the start of the path itself.

4.2.7. Trips are personal

```
fact uniqueTrips{
    all u1,u2: RegisteredUser|(
        all t: Trip|(
            u1 != u2 and
            (t in u1.registeredTrips) implies
            (t not in u2.registeredTrips)
        )
    )
}
```

Listing 4.13: Trips are personal

Since trips contain statistics that are relevant for each user, each Trip is unique to each user.

4.2.8. Path coherence

```
fact coherentPaths{
    // Segments are saved only if associated with another
    // segment or a Path (and at least an object Path
    // is needed in the chain)
    all s: Segment| (
        some ss: Segment|(
            ss.nextSegment = s or
            some p: Path|(
                s = p.start
            )
        )
    )

    all s: Segment|(
        // if segment is last in chain or in a cycle
        (s.nextSegment = none or s in s.^nextSegment) implies
        // then from said segment i can walk backwards
        // until i find a path's start
        some p: Path|(
            p.start in s.^~nextSegment
        )
    )

    // avoid self loops
    all s1, s2: Segment|(
        (s2 in s1.^nextSegment) implies (s1 not in s2.^nextSegment)

    )
}
```

Listing 4.14: Path coherence

Here we list the main rules for a path to be coherent: segment exist only if connected by another one (or a Path), if a Path connects with at least one of the same chain and loops are avoided.

4.2.9. Path equality

```
// show the segments of a path:
```

```
fun showPath[p: Path] : set Segment{
    p.start.*nextSegment
}

// collapse identical paths
fact pathEquality{
    // Paths are equal if the segments inside
    // are the same sequence of objects
    all p1,p2: Path|
        p1 != p2 implies (
            showPath[p1] != showPath[p2])
}
```

Listing 4.15: Path equality

Since different paths can be generated even when segments in them are the same in the same order, a function computes the sequence of segments of a path and a fact ensures that no two equal paths exist (so that trips can point to the same equal path without having duplicates).

4.3. Static Model Graph

4.3.1. Graphs and proof of consistency

```
run example{} for 20 but
    exactly 3 RegisteredUser,
    exactly 4 Trip,
    exactly 3 Path,
    exactly 6 Segment
```

Listing 4.16: First Run iteration

```
run example{} for 20 but
    exactly 5 RegisteredUser,
    exactly 6 Trip,
    exactly 1 Path,
    exactly 3 Segment
```

Listing 4.17: Second Run iteration

After all the signatures and facts we can run an example of an iteration and comment the results. From the figure below, we can see that by forcing the program to have less paths than trips Alloy gives us a consistent predicate by generating trips (in first run Trip3 and Trip0 for example) that

connect to the same path (which is exactly what we wanted in reality, since users can record different trips on the same path). Also, we notice that segments of different paths can point to the same next segment, which is allowed to optimize space and to not have multiple segments that model the same street. It is also possible that every user has no published trips (see second run). Below, the proof of consistency of the two run is also shown.

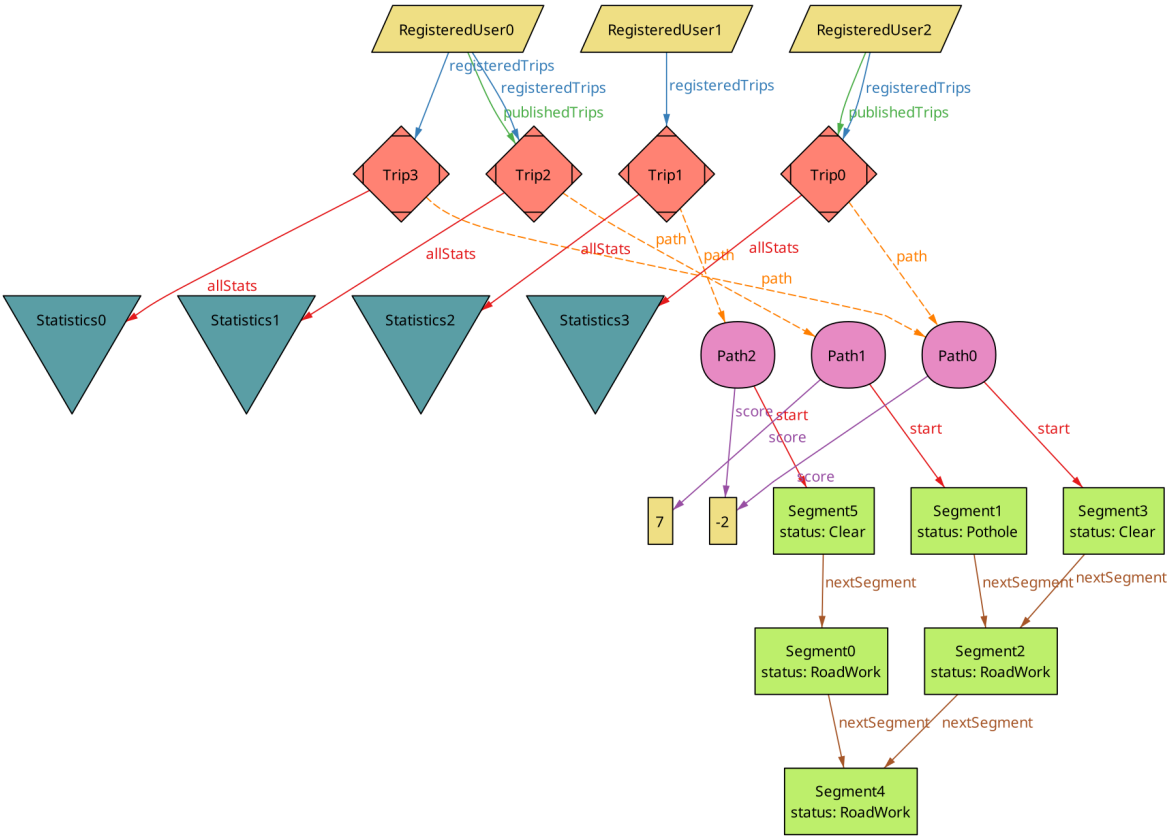


Figure 4.1: First Run result

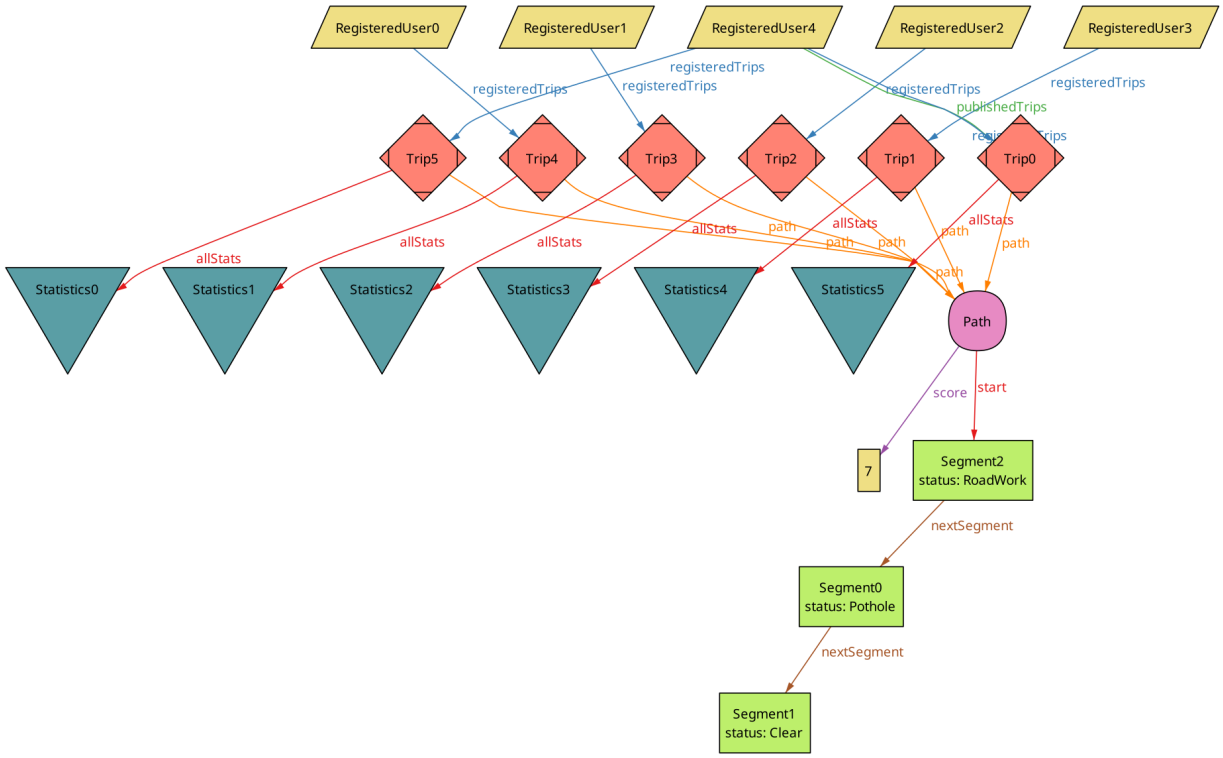


Figure 4.2: Second Run result

```
Executing "Run example for 20 but exactly 3 RegisteredUser, exactly 4 Trip, exactly 3 Path, exactly 6 Segment"
   Actual scopes: 3 Status, exactly 1 Clear, exactly 1 RoadWork, exactly 1 Pothole, exactly 6 Segment, exactly 3 Path, 20 Statistics, exactly 4 Trip,
   Solver=sat4j Bitwidth=4 MaxSeq=7 SkolemDepth=1 Symmetry=20 Mode=batch
   7110 vars. 396 primary vars. 10247 clauses. 71ms.
   Instance found. Predicate is consistent. 17ms.


Executing "Run example for 20 but exactly 5 RegisteredUser, exactly 6 Trip, exactly 1 Path, exactly 3 Segment"
   Actual scopes: 3 Status, exactly 1 Clear, exactly 1 RoadWork, exactly 1 Pothole, exactly 3 Segment, exactly 1 Path, 20 Statistics, exactly 6 Trip,
   Solver=sat4j Bitwidth=4 MaxSeq=7 SkolemDepth=1 Symmetry=20 Mode=batch
   7050 vars. 408 primary vars. 7235 clauses. 79ms.
   Instance found. Predicate is consistent. 19ms.
```

Figure 4.3: Proof of consistency

4.4. Merging of two paths

Since one of the goals of the application is the merging of paths that represent identical roads by updating their information based on an algorithm that judges their freshness, we decided to show how our model behaves in the instant of the merge. Our basic assumption is that, since Alloy doesn't permit deletions of entities once they're created, the merging of two paths is considered done when at least one of the two Path objects points to the starting Segment of the other Path object: what will be obtained is two paths that point to the same starting Segment, the remaining floating chain of segments (that for us will represent the "old" path, with outdated informations) is to be considered detached from the model.

4.4.1. Vars in signatures

```
sig Trip{
    tripStatistics: one Statistics,
    var path: one Path
}
```

Listing 4.18: New Path signature

To make the model mutable, we changed the Trip signature to make path a variable relation, in this way it will be possible to modify the path of a trip when the merging occurs.

4.4.2. Merge predicate

```
pred merge[u1: RegisteredUser, u2: RegisteredUser - u1]{
    one p1, p2: Path |(
        p1 in u1.registeredTrips.path and
        p2 in u2.registeredTrips.path and
        #(p1.start.^nextSegment) = #(p2.start.^nextSegment)

        // from now, p2 is the fresher path so p1 needs to
        // be disconnected
        // trips formerly on p1 now point to p2
        and all t: Trip | (
            ((t.path = p1) implies (t.path' = p2))
            and
            ((t.path != p1) implies (t.path' = t.path))
        )

        // statistics unchanged (new score is the same as p2)
    )
```

```
18 }
```

Listing 4.19: Merge predicate

In this predicate, we define the condition on which is possible to have a merge (paths must be of equal length) and how the merge is done: the new path of the newest trip is kept as it is, whereas the path of an old trip is changed to match the new one, disconnecting the old path from the entirety of the model (and not only for that specific trip) so that all trips connected to the old Path will connect to the new one.

4.4.3. Example predicate

```
1  pred mergingPathExample {
2      // at the start, at least a trip points to all paths
3      all p: Path|(
4          some t: Trip|(
5              p in t.path
6          )
7      )
8      // same for user - trips
9      all t: Trip|(
10         some u: RegisteredUser|(
11             t in u.registeredTrips
12         )
13     )
14     // give a feasible scenario
15     one u1, u2: RegisteredUser|(      // pick two users
16         some t1, t2: Trip|(
17             t1 in u1.registeredTrips and
18             t2 in u2.registeredTrips and
19             merge[u1,u2]
20         )
21     )
22 }
```

Listing 4.20: Merging example

By using a predicate to structure an example of merge, we defined a feasible situation where we can test the merge predicate (included at line 19). From the pictures below it is possible to notice how Trip1 loses the reference to Path1, acquiring the new Path0.

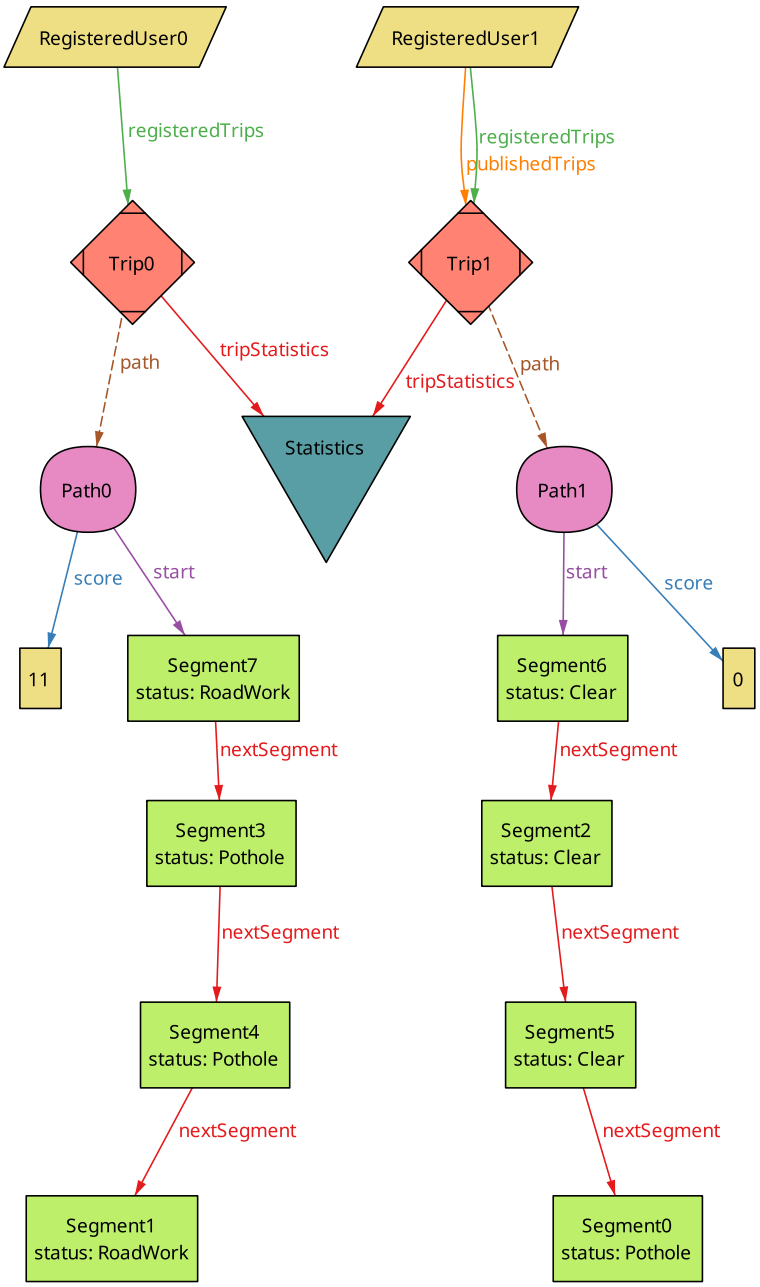


Figure 4.4: First instant of iteration

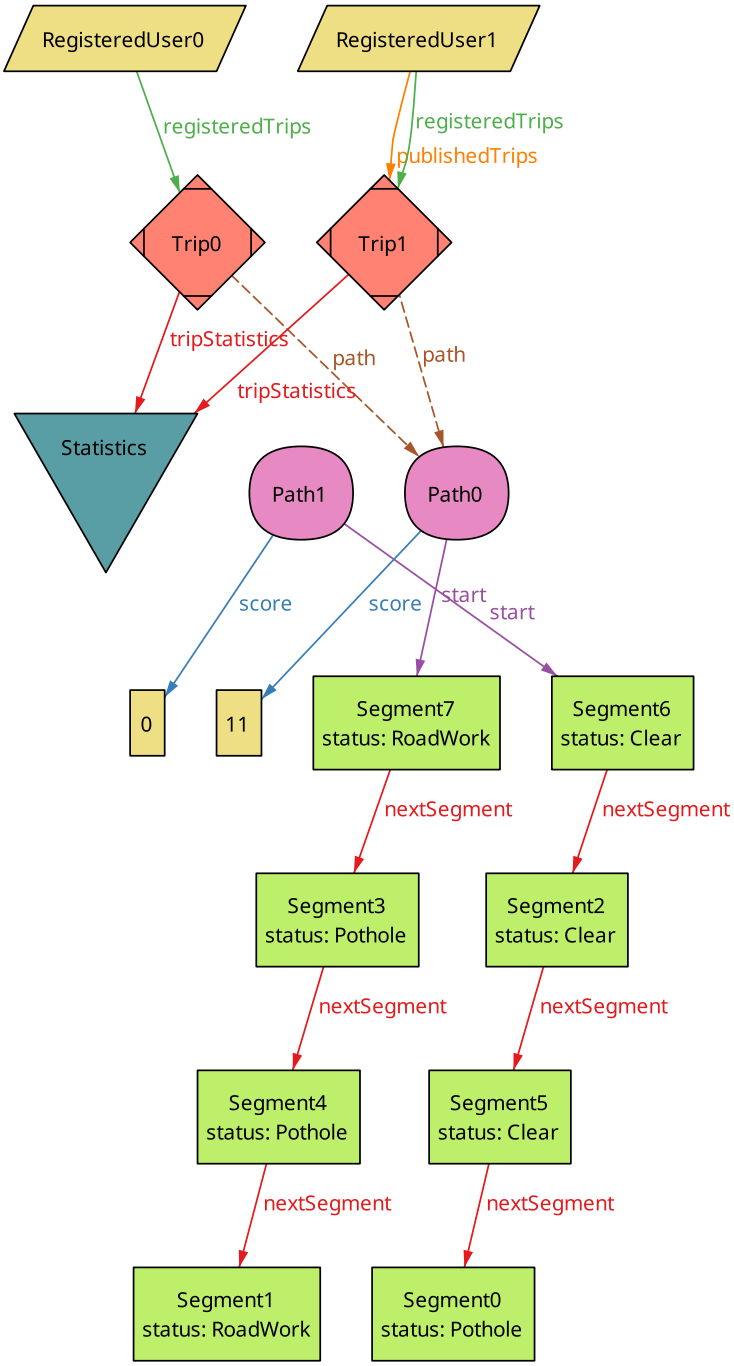


Figure 4.5: Second instant of iteration

```
Executing "Run mergingPathExample for 5 int, exactly 2 RegisteredUser, exactly 2 Trip, exactly 2 Path, exactly 8 Segment, 3 Statistics"
   Actual scopes: 3 Status, exactly 1 Clear, exactly 1 RoadWork, exactly 1 Pothole, exactly 8 Segment, exactly 2 Path, 3 Statistics, exactly 2 Trip, exactly 2 RegisteredUser
   Solver=sat4j Steps=1..10 Bitwidth=5 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
   1..2 steps. 9763 vars. 544 primary vars. 24577 clauses. 113ms.
   Instance   found. Predicate is consistent. 31ms.
```

Figure 4.6: Second instant of iteration

5 | Effort Spent

A table representing the total hours of work is shown here.

Work Areas	Alessandro Burato	Gabriele Crimella	Davide Faccini
Introduction	1h	2h	1h
Overall Description	1h	9h	1h
Specific Requirements	2h	2h	11h
Alloy	11h	1h	2h
Group Meetings and Reviews	8h	8h	8h

Table 5.1: Table of Effort

6 | References

Chapter 6

List of Figures

2.1 Class diagram representing the main entities and their relationships in the BBP system. 8
2.2 State diagram of the trip recording process. 9
2.3 State diagram of the trip recording process. 9
2.4 State diagram of the manual trip registration process. 10
2.5 State diagram of the path data aggregation process. 10

3.1 Use Cases 18
3.2 Sign up to BBP 21
3.3 Login to BBP 23
3.4 Logout to BBP 25
3.5 Recording with Manual Mode 28
3.6 Recording with Automated Mode 31
3.7 Personal Trip Storage 33
3.8 Search Path 35
3.9 Look Publish Path 37
3.10 Obstacles detection 39

4.1 First Run result 58
4.2 Second Run result 59
4.3 Proof of consistency 59
4.4 First instant of iteration 62
4.5 Second instant of iteration 63

List of Tables

1.1 List of Goals . . . 2
1.2 World Phenomena . . . 2
1.3 Machine Phenomena . . . 3
1.4 Shared Phenomena – World-Controlled . . . 3
1.5 Shared Phenomena – Machine-Controlled . . . 4

3.1 Requirements List . . . 17
3.2 Sign up to BBP . . . 20
3.3 Login to BBP . . . 22
3.4 Logout to BBP . . . 24
3.5 Recording with Manual Mode . . . 27
3.6 Recording with Automated Mode . . . 30
3.7 Personal Trip Storage . . . 32
3.8 Search Path . . . 34
3.9 Look Publish Path . . . 36
3.10 Obstacles detection . . . 38
3.11 Mapping Requirements on Goals . . . 45

5.1 Table of Effort . . . 65